

Multi-UAV Cooperative Pursuit Strategy With Limited Visual Field in Urban Airspace: A Multi-Agent Reinforcement Learning Approach

Zhe Peng , Guohua Wu , Senior Member, IEEE, Biao Luo , Senior Member, IEEE, and Ling Wang , Member, IEEE

Abstract—The application of multiple unmanned aerial vehicles (UAVs) for the pursuit and capture of unauthorized UAVs has emerged as a novel approach to ensuring the safety of urban airspace. However, pursuit UAVs necessitate the utilization of their own sensors to proactively gather information from the unauthorized UAV. Considering the restricted sensing range of sensors, this paper proposes a multi-UAV with limited visual field pursuit-evasion (MUV-PE) problem. Each pursuer has a visual field characterized by limited perception distance and viewing angle, potentially obstructed by buildings. Only when the unauthorized UAV, i.e., the evader, enters the visual field of any pursuer can its position be acquired. The objective of the pursuers is to capture the evader as soon as possible without collision. To address this problem, we propose the normalizing flow actor with graph attention critic (NAGC) algorithm, a multi-agent reinforcement learning (MARL) approach. NAGC executes normalizing flows to augment the flexibility of policy network, enabling the agent to sample actions from more intricate distributions rather than common distributions. To enhance the capability of simultaneously comprehending spatial relationships among multiple UAVs and environmental obstacles, NAGC integrates the “obstacle-target” graph attention networks, significantly aiding pursuers in supporting search or pursuit activities. Extensive experiments conducted in a high-precision simulator validate the promising performance of the NAGC algorithm.

Index Terms—Graph attention network, limited visual field, multi-agent reinforcement learning (MARL), normalizing flow, pursuit-evasion.

I. INTRODUCTION

THE rapid advancement of unmanned aerial vehicles (UAVs) has brought a significant revolution in various sectors, including mapping, aerial photography, and rescue

operations [1]. However, in some crucial or confidential urban zones, unauthorized UAVs are prohibited due to their potential threat to the airspace security. To counter the threats posed by unauthorized UAVs, using drone-jammer is often limited in flexibility or incurs high costs [2]. In recent years, the use of UAVs to counter unauthorized UAVs has emerged as an innovative strategy [3]. It provides greater maneuverability and safety, enabling the pursuit UAVs to swiftly approach unauthorized UAVs and utilize net guns or other tactics effectively to capture them. Moreover, deploying multi-UAV can enhance the efficiency of pursuit missions. Strategic coordination among these UAVs ensures the effective achievement of pursuit. Therefore, planning the movements of multiple pursuit UAVs to swiftly capture fast-moving unauthorized UAV is a problem with profound significance.

The pursuit-evasion game, rooted in the dynamics of predator-prey interactions in nature, is a classical problem extensively explored within the domains of robotics and cybernetics [4]. Planning pursuit UAVs to pursuit and capture unauthorized UAVs can similarly be conceptualized as a pursuit-evasion scenario. Pursuers should strategize their movements to approach the unauthorized UAV (evader) as soon as possible, encompassing it within their capture range. Conversely, the evader endeavors to maintain a considerable distance from pursuers to evade capture. However, most prevalent researches on pursuit-evasion often presume mutual access to each other's information by the pursuer and evader [5]. In reality, unauthorized UAVs do not voluntarily disclose their information to pursuit UAVs. In this scenario, pursuit UAVs rely on limited sensor data to complete the pursuit mission, thereby holding greater practical significance but posing heightened challenges.

With the progress in multi-sensor fusion technology, the remote observation and positioning of UAVs have become achievable [6]. However, the sensor perception range, i.e. visual field of pursuit UAVs, remains limited due to inherent limitations in sensors, such as distance or angle constraints. Based on this situation, we propose a multi-UAV with limited visual field pursuit-evasion (MUV-PE) problem. Compared to the previous pursuit-evasion game with multi-UAV [7], the most distinctive aspect of the MUV-PE problem lies in the pursuers' lack of real-time information about the evader. Pursuer must rely on a visual field with limited perception distance and viewing angle, which may be obstructed by build-

Manuscript received August 2, 2024; revised September 24, 2024; accepted October 2, 2024. This work was supported in part by the National Natural Science Foundation of China (62373380). Recommended by Associate Editor Zongyu Zuo. (Corresponding author: Guohua Wu.)

Citation: Z. Peng, G. Wu, B. Luo, and L. Wang, “Multi-UAV cooperative pursuit strategy with limited visual field in urban airspace: A multi-agent reinforcement learning approach,” *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 7, pp. 1350–1367, Jul. 2025.

Z. Peng is with the School of Traffic and Transportation Engineering, Central South University, Changsha 410017, China (e-mail: peng-zhe@csu.edu.cn).

G. Wu and B. Luo are with the School of Automation, Central South University, Changsha 410017, China (e-mail: guohuawu@csu.edu.cn; biao.luo@csu.edu.cn).

L. Wang is with the Department of Automation, Tsinghua University, Beijing 100084, China (e-mail: wangling@tsinghua.edu.cn).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/JAS.2024.124965

ings, to actively gather information of the evader. Pursuers can only obtain the position information of the evader when it enters the visual field of any pursuer, while other details such as speed and yaw remain inaccessible. If the evader moves out of the visual field of all pursuers, the pursuers will lose the information about it. In this problem, our objective is to seek a pursuit strategy for the pursuers, enabling them to efficiently capture the higher-speed evader rather than merely observe it [8]. At the same time, pursuers should avoid collisions. Therefore, these characteristics turn the MUV-PE problem into a multifaceted challenge encompassing elements of “search-pursuit-obstacle avoidance”.

The MUV-PE problem is a special variant of the pursuit-evasion game, and conventional solutions to the pursuit-evasion are differential game and cooperative control theories [9]. In numerous past studies, the pursuit-evasion game was formulated as a differential game [10]. The optimal strategies for pursuers were derived by directly solving the Hamilton-Jacobi-Isaacs (HJI) equation. Concurrently, some studies utilized cooperative control theories to coordinate pursuers based on the strategies provided by experts or geometric analysis [11]. These aforementioned solutions are mostly based on the known dynamics of the environment. However, in many realistic problems, due to the complexity and uncertainty of the environment, modeling the dynamics could be nearly impossible [12]. Additionally, in the MUV-PE problem, pursuers must also consider searching for or reacquiring (previously in sight, now lost) the moving evader. Moreover, the evader can take actions to avoid being detected by pursuers, which presents challenges for some classic coverage search algorithms [13]. Consequently, solving this problem by traditional mathematical methods becomes exceptionally challenging and, in some cases, unfeasible.

To address these challenges, the learning-based approach is emerging that leverages reinforcement learning (RL) to tackle the pursuit-evasion game amidst complex and unknown dynamics [14]. This method obviates the need for circumventing the daunting problem of modeling environmental dynamics. RL enables the fitting of appropriate control strategies or the adjustment of controller parameters via neural networks [15], thus allowing the agent to autonomously learn motion strategies through continuous interaction with the environment. Furthermore, the learning-based method exhibits stable adaptability even when environment changes, in contrast to traditionally designed controllers which may perform poorly in new environment settings [16]. When there are multiple pursuers, the learning-based approach typically employs multi-agent reinforcement learning (MARL) to coordinate their actions effectively [17].

Based on the aforementioned analysis, an MARL-based approach emerges as a promising and effective way for solving the MUV-PE problem. Consequently, this paper focuses on the characteristics of the MUV-PE problem, and introduces the normalizing flow actor with graph attention critic (NAGC) algorithm. NAGC adopts the Soft Actor-Critic architecture [18] and extends it to a centralized training with decentralized execution framework for multi-agent systems. NAGC introduces two specific advancements. Firstly, traditional non-

deterministic continuous MARL algorithms typically use the Gaussian distribution sampling to generate actions [19], [20]. In contrast, the Actor network in NAGC further maps the sampled actions from the Gaussian distribution through normalizing flows, making them equivalent to actions sampled from a more complex distribution. This approach mitigates the flexibility limitations caused by the unimodal nature of Gaussian distributions. Additionally, the normalization flow can be updated in an end-to-end manner within the MARL, training the Actor network to realize the required distribution form. Secondly, most MARL algorithms based on the Actor-Critic architecture typically employ multilayer perceptrons to build the centralized training network [21], [22]. However, NAGC incorporates an “obstacle-target” graph attention network into the Critic network. This enables the Critic network to effectively process and learn from graph-type data, significantly enhancing its ability to capture spatial features among UAVs and obstacles. This improvement helps to form more effective movement patterns among the pursuers, enhance the coordination of capturing the evader and maintain a better visual coverage. Our main contributions include:

- 1) We proposed an MUV-PE problem where pursuers need to search and capture evader with a limited visual field, while avoiding collision.
- 2) We applied normalizing flows to enhance the flexibility of policy network in NAGC, which is capable of sampling actions from a more precise distribution.
- 3) We integrated an “obstacle-target” graph attention network to the Critic network in NAGC, to capture spatial features among UAVs and obstacles.
- 4) We conducted numerous experiments in a high-precision simulator, demonstrating the promising performance of NAGC algorithm.

The remainder of this paper is organized as follows: Section II reviews the related work on pursuit-evasion and MARL. Section III formulates the MUV-PE problem. Section IV constructs the Markov decision process and elaborates our NAGC approach. Section V conducts extensive experiments and result analyses. Section VI concludes the paper.

II. RELATED WORK

A. Pursuit-Evasion

The pursuit-evasion was initially introduced by Isaacs in his book, *Differential Games*, and has since become a subject of extensive investigation [23]. In 1976, Parsons [24] illuminated the pursuit-evasion game within a finitely connected graph, offering the example of multiple individuals rescuing trapped people. It presented a tangible depiction of the pursuit-evasion. Subsequently, Oyler *et al.* [25] considered obstacles that impede the players’ movements in the planar pursuit-evasion game. Taking inspiration from the group hunting and foraging behaviors of predators, Chen *et al.* [11] explored a scenario with multi-pursuer and a higher speed evader. As theories of the pursuit-evasion progressed, researchers began applying them into practical scenarios like UAV pursuit [5].

The visibility-based pursuit-evasion problem represents a

specialized case within the pursuit-evasion, originally introduced by Suzuki and Yamashita [26]. In this scenario, the motion of one or more pursuers is strategically planned within a polygonal environment. The pursuer possesses an infinite-length visual field, but obstructed by walls. The primary objective for the pursuer is to visually detect an evader who is unpredictable, has an unknown initial position, and can move at any speed. Subsequently, a variant known as the ϕ -searcher version emerged [27], restricting the pursuer's visual field to a specific angle range. Other versions involve constraints on the speeds of both the pursuer and the evader [28] or operate within an unknown planar environment [29]. However, owing to the intricate nature of the visibility-based pursuit-evasion problem, many experimental scenarios are limited in scale, often involving small polygonal environments or grid diagrams [30]. In most research cases, the success criterion for the pursuer is typically achieving visual contact or maintaining continuous visual contact, without the necessity to physically capture the evader by close proximity and "touching" it.

Differential game theory stands out as a primary approach in tackling the pursuit-evasion game [4]. This methodology integrates the differential equations governing the players' movements and directly addresses the HJI differential equation [9]. In three-dimensional space scenarios, Chen *et al.* [10] derived equilibrium strategies by solving the retrogressive path equation according to the HJI method. However, this approach is overly complex for multiplayer scenarios, designing motion strategies through geometric analysis is an important method currently [31]. For the visibility-based pursuit-evasion problem, it is more about incorporating the characteristics of the environment, combining dominant strategies with optimal control or geometric approaches, however, these methods are only applicable to very simple environments. Currently, there are also sampling-based methods [32], but they still face significant challenges in design and solution when dealing with complex environments and multiplayer scenarios. Any alterations in the environment's shape or size may significantly compromise the effectiveness of previously strategies.

B. MARL and GAT

In recent years, MARL has garnered significant attention and achieved advancements in various fields, such as path planning [33], UAV-assisted communication [34] and dynamic algorithm configuration [35]. MARL is particularly adept at handling environmental uncertainty and learning to solve unknown tasks without requiring extensive domain knowledge [36]. In MARL, a team of agents coexists within the same environment, each independently interacting with it. Compared to single-agent RL, MARL offers robust support for modeling and solving many large-scale and complex problems. Enhancing the performance of algorithms to achieve stronger coordination has always been a critical direction in MARL development. Researchers have designed numerous algorithms for this purpose, including policy gradient-based methods such as MAPPO [19], and value-based methods such as QMIX [37]. These methods have demonstrated excellent cooperative abilities in many tasks.

Moreover, since spatial relationships between players are crucial in the pursuit-evasion game, it is essential to enhance the ability of learning-based methods to explore and learn spatial features. Graph neural networks (GNNs) have become the state-of-the-art model for processing graph-structured data [38]. Graph attention networks (GATs) incorporate attention mechanisms into GNNs, enabling nodes to pre-calculate the importance of neighboring node representations and perform neighborhood aggregation based on the learned importance coefficients [39]. GATs have seen many novel developments, such as heterogeneous GATs [40] and sparse GATs [41], making them adaptable to various tasks. Some current algorithms combine GATs with MARL, effectively strengthening spatial cooperation among agents [42]. However, most of them tend to overlook the potential spatial relationships between agents and environmental features (such as obstacles or communication base stations).

C. MARL in Pursuit-Evasion

The advent of RL has introduced a novel approach to solving the pursuit-evasion, commonly referred to as the learning-based method [14]. In recent years, more and more work has begun to consider using MARL methods for pursuit-evasion problems, demonstrating the effectiveness of learning-based methods. MARL methods bypass the need for extensive modeling and analysis of complex and even dynamic environments and multiple players. Instead, they achieve collaborative capture of the evader by multiple pursuers in an end-to-end manner driven by reward signals. Fu *et al.* [16] utilized a decomposed multi-agent deep deterministic policy gradient algorithm to address the multi-UAV pursuit-evasion. Zhang *et al.* [5] explored the pursuit-evasion with multi-UAV in a two-dimensional urban airspace, devising a multi-agent coronal bidirectionally coordinated with the target prediction algorithm. Du *et al.* [43] delved into the pursuit-evasion with multi-UAV in a two-dimensional multi-obstacle urban airspace, considering communication constraints among UAVs.

In light of the preceding analysis, most studies on the visibility-based pursuit-evasion problem primarily focus on visually observing the evader, without the necessity of physical capture. Moreover, lots of pursuit-evasion scenarios are usually based on the assumption that pursuers know the information about evader in real-time. In the MUV-PE problem, in addition to using their own limited visual field to actively obtain the information of the evader, pursuers must also "touch" the evader closely to achieve mission success, going beyond mere visual detection. By employing learning-based methods, the pursuers can autonomously learn pursuit strategies in the environment. Given the unique characteristics of the MUV-PE problem, we propose an enhanced MARL algorithm, offering a novel avenue for achieving intelligent and autonomous motion-planning in the pursuit-evasion with multi-UAV.

III. PROBLEM FORMULATION

In this section, we will outline the problem scenario and the kinetics of UAV, and subsequently emphasize the primary

challenge: The limited visual field. Finally, we will proceed with modeling the objective and constraints of the problem.

A. Scenario Description

In a confined two-dimensional urban airspace, containing multiple buildings, there are N identical pursuit UAVs with the goal of collaborating to capture an evasion UAV. Each pursuer has a limited visual field characterized by restricted length and viewing angle, which can be obstructed by buildings. Only position of the evader can be obtained by pursuers when it is within the visual field of at least one pursuer. Other details such as the evader's speed and yaw angle are inaccessible to the pursuers. Once the evader moves beyond the collective visual fields of all pursuers, its whereabouts become unknown to them. Assume that pursuers maintain the same altitude as the evader. Furthermore, all UAVs need to avoid collision with buildings and are prohibited from leaving the designated airspace. Fig. 1 depicts a simple scenario of MUV-PE problem. In this problem, the evader's speed surpasses those of the pursuers. The evasion policy employed by the evader remains undisclosed to the pursuers. In this problem, we have made the following assumptions:

- 1) Communication between the pursuit UAVs is normal throughout the entire area and time period.
- 2) Pursuit UAVs can always obtain information about their own coordinates, speed, and attitude angles.
- 3) Once the evasion UAV enters the visual field of any pursuit UAV, its coordinates can be determined by them. Delays and errors in locating the evasion UAV are not considered.
- 4) All UAVs are at the same altitude. When the distance between the evasion UAV and any pursuit UAV is less than the capture radius, it is considered a successful capture.

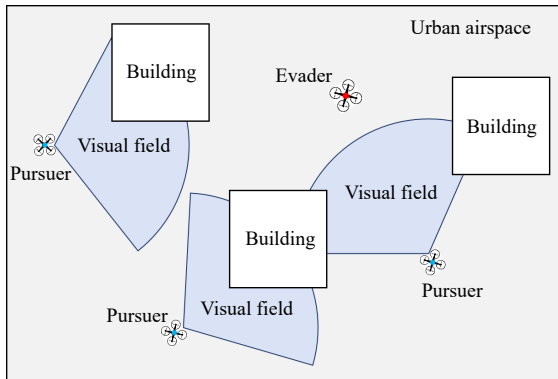


Fig. 1. A simple scenario of the MUV-PE problem.

B. UAV Kinetics

In the context of the two-dimensional airspace scenario, the coordinates of pursuer $i \in \{1, 2, \dots, N\}$ at time t are denoted as $\mathbf{p}_i^t = [x_i^t, y_i^t]$. Similarly, the coordinates of the evader, represented by e , are denoted as $\mathbf{p}_e^t = [x_e^t, y_e^t]$. x^t and y^t respectively denote the horizontal and vertical coordinates of the UAV at time t . The velocity magnitude and yaw angle of the UAV at time t are denoted as v^t and ψ^t , respectively.

Assume that all pursuit UAVs have the same identical kinetics model. The equations of kinetics are

$$\begin{cases} \dot{x}^t = v^t \cos \psi^t \\ \dot{y}^t = v^t \sin \psi^t \\ \dot{\psi}^t = \omega^t \\ \dot{v}^t = a^t \end{cases} \quad (1)$$

where ω^t denotes the UAV's angular velocity, a^t denotes the UAV's acceleration. We will use RL methods to generate the desired ω^t and a^t . Due to UAV performance constraints, the acceleration a and angular velocity ω have certain upper and lower limits

$$\begin{aligned} -\omega_{\max} &\leq \omega^t \leq \omega_{\max} \\ -a_{\max} &\leq a^t \leq a_{\max} \end{aligned} \quad (2)$$

where $\omega_{\max}$ denotes the maximum angular velocity of the UAV, and $a_{\max}$ denotes the maximum acceleration of the UAV.

C. Limited Visual Field of Pursuers

Each pursuer has a limited visual field, usually facilitated by on-board sensors such as telephoto cameras, radar, and similar technologies used in real-world applications. The visual fields of all pursuers are identical. In this paper, the visual field $\mathcal{V}_i$ of pursuer i can be modeled as a sector-shaped area

$$\mathcal{V}_i = \{\mathbf{p} | \mathbf{m}_{i,k}^t \mathbf{p} \leq n_{i,k}^t, \|\mathbf{p} - \mathbf{p}_i^t\|_2 \leq L, k = 1, 2\} \quad (3)$$

where $\mathbf{m}_{i,1}^t, \mathbf{m}_{i,2}^t \in \mathbb{R}^2$, $n_{i,1}^t, n_{i,2}^t \in \mathbb{R}$ form the two linear boundaries that limit the viewing angle φ . This representation shows that the area covered by the visual field changes with the movement of the pursuer. L is the perception distance, constraining the detection range. Both φ and L are fixed constants, corresponding to the arc angle and radius of the sector respectively. The pursuer's sensor is fixed in the forward direction, so the direction of the pursuer's yaw angle ψ divides the visual field into two equal parts, as illustrated in Fig. 2. It is worth noting that the pursuer's visual field can be obstructed by obstacles. If a point's line of sight to the pursuer is interrupted by an obstacle, that point is considered invisible. This means the actual visible range $\mathcal{V}_i^r$ is a subset of $\mathcal{V}_i$. We have also included examples of such invisible points in Fig. 2. A way of judging whether the evader is visible is included in S1 of Supplementary Material¹.

D. Objective and Constraints

Assume that all UAVs are confined to movement within a finite airspace denoted as Ω . We define the set of buildings in this area, which are taller than the flying height of the UAVs, as B . Each building is assumed to be cuboid in shape, with its horizontal cross-section in the 2D plane being a rectangle. The length and width of the rectangle are parallel to the x -axis and y -axis of the 2D coordinate system respectively. For a building b , its center coordinates can be denoted as $\mathbf{p}_b = [x_b, y_b]$, its length as l_b , and its width as w_b . The area occupied by all buildings can be represented as Ω_B , which is a subset of the airspace Ω . The buildings are completely stationary and

¹ Supplementary Material of this paper can be found in link <https://www.ieee-jas.net/article/doi/10.1109/JAS.2024.124965?pageType=en#Supplements>.

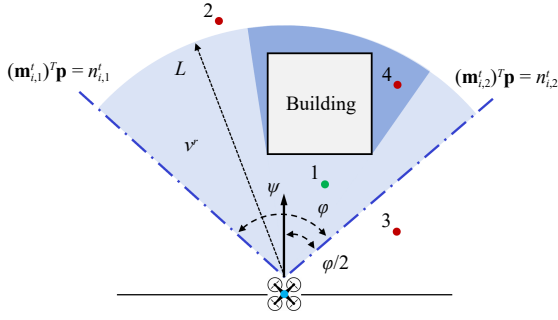


Fig. 2. Limited visual field of pursuers. When the evader is positioned at points 2, 3, and 4, the pursuers' vision is restricted due to the perception distance, viewing angle, and obstruction caused by a building, consequently impeding the detection of the evader. However, when the evader is at position 1, it falls within the detection range of the pursuer. Simultaneously, the pursuer will share the evader's coordinate information with other pursuers.

obstruct the pursuer's visual field. Additionally, the buildings also impede UAV flight.

Assume that the initial coordinates of all pursuers are fixed, while the initial coordinates of the evader are randomly located in any non-building area $\Omega_e = \Omega \setminus \Omega_B$. The pursuers aim to approach the evader closely and capture it. It is assumed that each pursuer possesses a circular capture range, centered at its own position, with a radius denoted by δ_{cap} . When the evader enters the capture range of any pursuer, it is considered a successful capture. Hence, the condition for a successful capture can be defined as follows:

$$\|\mathbf{p}_i^t - \mathbf{p}_e^t\|_2 \leq \delta_{cap}, \exists i \in P. \quad (4)$$

We denote the time of achieving successful capture as T_c . It is also interpreted as the total duration spent successfully completing the mission. Additionally, there is a maximum time T_{max} set for the mission.

For pursuer i , it utilizes information $\mathbf{o}_i^t$ obtained from its sensors and executes actions $\mathbf{u}_i^t = [\omega_i^t, a_i^t]$ that adjust the angular velocity ω_i^t and acceleration a_i^t to capture the evader. Hence, the control policy of pursuer i can be denoted as $\pi_i(\mathbf{u}_i^t | \mathbf{o}_i^t)$. We denote the joint control policies of all pursuers as Π_{jt} , and hope to get a certain joint control policy $\pi_{jt} = [\pi_1, \pi_2, \dots, \pi_N] \in \Pi_{jt}$ to expedite the capture of the evader efficiently. This problem can be formulated as follows:

$$\arg \min_{\pi_{jt}} \mathbb{E}[T_c | \Pi_{jt} = \pi_{jt}] \quad (5)$$

$$\text{s.t. } |x_i^t - x_b| \geq l_b/2, \forall i \in P, \forall b \in B, \forall t \in \{1, 2, \dots, T_c\} \quad (6)$$

$$|y_i^t - y_b| \geq w_b/2, \forall i \in P, \forall b \in B, \forall t \in \{1, 2, \dots, T_c\} \quad (7)$$

$$\|\mathbf{p}_i^t - \mathbf{p}_j^t\|_2 \geq \delta_s, \forall i, j \in P, i \neq j, \forall t \in \{1, 2, \dots, T_c\} \quad (8)$$

$$\mathbf{p}_i^t = \mathbf{p}_i^{t-1} + \Delta t \mathbf{u}_i^{t-1}, \forall i \in P, \forall t \in \{1, 2, \dots, T_c\} \quad (9)$$

$$\mathbf{p}_i^t \in \Omega, \forall i \in P, \forall t \in \{1, 2, \dots, T_c\} \quad (10)$$

$$0 \leq T_c \leq T_{max} \quad (11)$$

where (6) and (7) show that the pursuers need to avoid collisions with any buildings within the area; (8) shows that the pursuers need to avoid collisions with each other, maintaining a minimum safe distance δ_s between any two pursuers; (9) shows that the pursuit UAV needs to change its coordinates in discrete time by executing action $\mathbf{u}_i$, adhering to the UAV kinetics; (10) shows that the pursuers are confined within the designated airspace; (11) shows that the necessity of completing the mission within the predetermined maximum mission time. A brief analysis about solution existence based on the Apollonius circle is included in S2 of Supplementary Material¹.

IV. METHODOLOGY

In this section, we have decomposed the MUV-PE problem into phases, and build a decentralized partial observable Markov decision process (Dec-POMDP) for the problem. Based on it, we propose an NAGC algorithm and introduce its architecture, training algorithm.

A. Phase-Based Processing

Considering the limited visual field of pursuers presents two notable challenges. Initially, the evader might not fall within the pursuers' visual field, necessitating collaborative efforts for pursuers to search the evader without any prior information. Due to the obstruction of the visual field by environmental obstacles and the rapid movement of evader, even if the pursuers detect the evader, maintaining visual contact becomes challenging over time.

In this context, considering the pursuer's perspective, we have initially delineated three phases termed as "search-pursuit-reacquisition" based on two criteria: "Whether the evader has previously been detected" and "whether the evader is currently within the visual field". When the evader has not been previously detected, the current phase of the pursuers is categorized as the search phase. This phase requires pursuers to collectively search for the evader. When the evader is currently within the visual field, we define the pursuers' current phase as the pursuit phase. Here, the primary objective is to collaboratively pursuit and capture the evader. Lastly, when the evader has been previously detected but is currently not within the visual field of the pursuers, we categorize the pursuers as being in the reacquisition phase. This phase necessitates collective actions to find back the evader, re-establishes visual contact and returns to pursuit phase.

However, the reacquisition phase possesses certain distinctive characteristics. In contrast to the search phase, during the reacquisition phase, pursuers have previously detected the evader, which eliminates the possibility that the evader could be present in any position within the airspace. Upon the evader's disappearance, we designate the last observed point of evader, denoted as β , as the "lost point". The temporal interval between the evader's lost moment and the current time is termed the "lost time", denoted as t_{lost} . Consequently, the evader must be situated within a circular expanse centered at β , with a radius of $t_{lost} \times v_{e,max}$, where $v_{e,max}$ represents the evader's maximum speed. This region is referred to as the "suspect area".

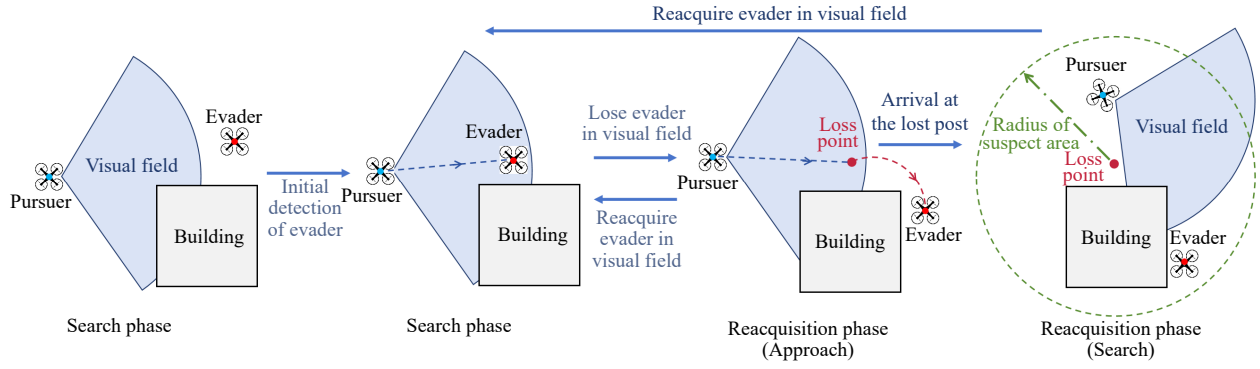


Fig. 3. “Search-pursuit-reacquisition (Approach)-Reacquisition (Search)” phases. The transition from the search phase to the pursuit phase is unidirectional. Once the evader is detected, there will be no further search phase within that episode. However, the connection between pursuit phase and reacquisition phase is bidirectional because after detecting the evader, there is a possibility of losing evader within the visual field. When the evader re-enters the visual field, the transition reverts from the reacquisition phase back to the pursuit phase.

In this problem, it is not merely about sighting the evader, and we also require sufficient proximity for capture. Similarly to the pseudo-blocking vertex in [44], we employ a convenient and feasible method to facilitate the efficient reacquisition of the evader in this paper. Upon the evader’s loss, we encourage the pursuers to promptly reach the lost point. This is because during this initial phase, the suspect area remains relatively small, causing that the probability of reacquiring the evader near the lost point is significantly high. Furthermore, this process inherently pushes the pursuers to advance closer to the evader. Consequently, we designate the duration from the moment pursuers lose visual contact with the evader until their arrival at the lost point as the “reacquisition phase (approach)”. If, en route to the lost point, any pursuer detects the evader again, the reacquisition phase (approach) concludes prematurely, reverting to the pursuit phase. Once a pursuer reaches the lost point without any pursuer having sighted the evader during this process, they will initiate a search within the suspect area. We term this phase the “reacquisition phase (search)”. If the evader is rediscovered during this phase, the reacquisition phase (search) ends, reverting to the pursuit phase.

Hence, we expand the “search-pursuit-reacquisition” phases to the “search-pursuit-reacquisition (approach)-reacquisition (search)” phases. Fig. 3 illustrates the interconversion process among these phases.

B. Dec-POMDP

Dec-POMDP is always taken as the standard for modeling fully cooperative multi-agent tasks, as a tuple $\mathcal{G} = \langle S, U, \mathcal{P}, r, O, N, \gamma \rangle$, consisting of the following components: State space S , action space U , state transition function $\mathcal{P}$, reward function r , observation space O , number of agents N , discount factor γ . Here each pursuer is treated as an individual agent.

1) *Action*: The action of agent $i \in \{1, 2, \dots, N\}$ at time t can be defined as $\mathbf{u}_i^t \in U$. The action space U is continuous, with each agent executing a two-dimensional action, denoted as $\mathbf{u}_i^t = [\omega_i^t, a_i^t]$, encompassing the angular speed and acceleration of the pursuer i . The actions of all agents at the same time can be combined into joint action $\mathbf{u}_{jt}^t = [\mathbf{u}_i^t]_{i=1}^N$. The joint action $\mathbf{u}_{jt}^t$ is treated as the output of the pursuers system.

2) *Local Observation*: The local observation of agent $i \in \{1, 2, \dots, N\}$ at time t can be defined as $\mathbf{o}_i^t \in O$, and each agent has its own individual $\mathbf{o}_i^t$. According to the limited visual field, agents’ perception of the environment is partially observable. Each agent’s observation $\mathbf{o}_i^t$ consists of three parts (For better understanding, please refer to S3 of Supplementary Material¹):

a) *Agent’s own state information*: This includes the agent’s own coordinates $\mathbf{p}_i^t$, yaw angle ψ_i^t , velocity $\mathbf{v}_i^t$ (the x -direction, y -direction velocities) and obstacle information $\mathcal{L}_i^t$. For $\mathcal{L}_i^t$, UAV can emit radar rays from five directions: Front, left/right 30 degrees, and left/right 60 degrees, with a maximum length of $\lambda_{\max}$ to perceive obstacles. As the UAV nears an obstacle, certain radar rays will be obstructed, resulting in reduced lengths. The UAV leverages the ratio of the actual returned lengths of the five radar rays to $\lambda_{\max}$ as the obstacle information, denoted by $\mathcal{L}_i^t = [\mathcal{L}_{i,1}^t/\lambda_{\max}, \mathcal{L}_{i,2}^t/\lambda_{\max}, \dots, \mathcal{L}_{i,5}^t/\lambda_{\max}]$.

b) *Teammates’ state information*: This includes the relative coordinates $\Delta \mathbf{p}_{ij}^t$, distance d_{ij}^t , relative yaw angle $\Delta \psi_{ij}^t$, and azimuth angle ρ_{ij}^t of all allied agents, where $j \in \{1, 2, \dots, N\}$ and $i \neq j$.

c) *Evader information*: Since the evader is not always within the pursuers’ visual field, the pursuers may not always have real-time information about the evader. Therefore, the evader’s information is determined based on the phases of the pursuers:

i) During the pursuit phase, evader information includes the relative coordinates $\Delta \mathbf{p}_{ie}^t$, distance d_{ie}^t , and relative azimuth ρ_{ie}^t between the evader e and pursuer i . Evader information is all calculated based on the coordinates of the pursuer i and the evader e .

ii) During the reacquisition phase (approach), the focus shifts to the latest lost point β of the evader. Thus evader information includes the relative coordinates $\Delta \mathbf{p}_{i\beta}^t$, distance $d_{i\beta}^t$, and relative azimuth $\rho_{i\beta}^t$ between the lost point β and pursuer i .

iii) During the search phase or reacquisition phase (search), the evader information is replaced by zero.

Reward: In Dec-POMDP, all pursuers collaborate to optimize a unified reward signal known as the team reward. If set-

ting the reward for successful capture to 1 and unsuccessful capture to 0, a significant challenge known as the sparse reward problem will make learning difficult and exploration aimless [45]. To address this problem, we propose a differentiated phase reward function tailored to the various phases of pursuers. During the phases where the pursuers need to search for the evader, i.e., the search phase and reacquisition (search) phase, we did not include factors directly guiding the agent's actions in the reward functions. This was primarily to enhance and encourage the agent's exploration of the environment. During the phases where capturing the evader is necessary, i.e., the pursuit phase and reacquisition (approach) phase, reducing the distance to the evader or maintaining visual contact is crucial. Therefore, the reward functions in these two phases are formulated based on these two aspects. The agents need to select the corresponding reward function based on the current phase, thereby exploring in a reasonable direction to achieve the goal of each phase.

In the search phase, our goal is to expand the total visual field area covered by all pursuers within the airspace. It means that pursuers need to reduce the overlap area and the area outside the airspace within their visual fields. Therefore, the visual field area reward r_{s1}^t can be calculated using the following formula:

$$r_{s1}^t = \begin{cases} \mathcal{A}_c^t / C_1, & \mathcal{A}_c^t \geq \mathcal{A}_c^{t-1} \\ -\mathcal{A}_c^t / C_1, & \mathcal{A}_c^t < \mathcal{A}_c^{t-1} \end{cases} \quad (12)$$

where C_1 represents a constant, $\mathcal{A}_c^t$ denotes the total area covered by the visual field of pursuers within the airspace at time t .

Simultaneously, we encourage the pursuers to explore more previously uncovered areas within their visual range. Therefore, the reward r_{s2}^t for exploring new areas can be calculated using the following formula:

$$r_{s2}^t = (\mathcal{A}_n^t - C_2) / C_3 \quad (13)$$

where C_2 and C_3 are constants, $\mathcal{A}_n^t$ represents the area of the newly explored regions within the visual field of all pursuers at time t .

Hence, in the search phase, the phase reward r_s^t for pursuers can be obtained by adding r_{s1}^t and r_{s2}^t together.

In the pursuit phase, our aim is to make the pursuers swiftly approach and capture the evader. Hence, the distance between the pursuers and the evader is crucial in formulating the reward function. Therefore, the distance reward r_{p1}^t can be calculated using the following formula:

$$r_{p1}^t = \begin{cases} \sum_{i=1}^N (C_4 / d_{ie}^t), & d_{ie}^t \geq d_{ie}^{t-1} \\ -\sum_{i=1}^N (C_4 / d_{ie}^t), & d_{ie}^t < d_{ie}^{t-1} \end{cases} \quad (14)$$

where C_4 is a constant, d_{ie}^t represents the straight-line distance between pursuer i and evader e at time t .

In conjunction with the UAV kinetics, when the yaw angle of the pursuer is directed towards the evader as closely as possible, it allows for a quicker approach to it. Simultaneously, it

allows the pursuer to maintain a relatively stable visual contact with the evader. Therefore, incorporating yaw angles of pursuers, a corresponding yaw angle reward r_{p2}^t is calculated by the following formula:

$$r_{p2}^t = \sum_{i=1}^N C_5 \cos(\psi_i^t - \rho_{ie}^t) \quad (15)$$

where C_5 is a constant, ψ_i^t is the yaw angle of pursuer i , and ρ_{ie}^t is the relative azimuth between pursuer i and evader e . Hence, in the pursuit phase, the phase reward r_s^t for pursuers can be obtained by adding r_{p1}^t and r_{p2}^t together.

In the reacquisition phase (approach), the goal is to make the pursuers promptly reach the lost point. Hence, the reward function closely mirrors that of the pursuit phase. The adjustment required involves replacing the distance d_{ie}^t relative to the evader e with the distance $d_{i\beta}^t$ relative to the lost point β , and replacing the relative azimuth angle ρ_{ie}^t to the evader e with the relative azimuth angle $\rho_{i\beta}^t$ to the lost point β . So the phase reward in the reacquisition phase (approach) is not reiterated here.

In the reacquisition phase (search), we incentivize the pursuers to achieve a higher percentage of visual field coverage within the suspect area. Hence, the reward function can be calculated based on the ratio between the area of visual field covered by all pursuers within the suspect area $\mathcal{A}_s^t$ and the total area of the suspect area. It can be represented by the following formula:

$$r_s^t = \begin{cases} \mathcal{A}_s^t / [\pi \times (t_{\text{lost}} \times v_{e,\text{max}})^2], & \mathcal{A}_s^t \geq \mathcal{A}_s^{t-1} \\ -\mathcal{A}_s^t / [\pi \times (t_{\text{lost}} \times v_{e,\text{max}})^2], & \mathcal{A}_s^t < \mathcal{A}_s^{t-1} \end{cases} \quad (16)$$

where t_{lost} is the loss time of evader, $v_{e,\text{max}}$ is the maximum speed of evader.

Therefore, the rewards obtained by the pursuers are tailored based on their current phase, aiming for effective goal achievement in each phase. As the obstacle avoidance strategy of the pursuers requires continual learning during interactions with the environment, an additional collision warning reward r_c^t is introduced. Specifically, when a pursuer approaches an obstacle closely, it receives a negative collision warning reward. The warning distance is δ_w , and the reward is calculated proportionally closer to the obstacle

$$r_c^t = \sum_{i=1}^N \min(C_6 \times (\delta_w - \lambda_{i,1}^t), 0) \quad (17)$$

where C_6 is a negative constant, $\lambda_{i,1}^t$ represents the returned length of the radar ray directly in front of pursuer i at time t .

Therefore, as the mission progresses, the team reward shared by all agents is denoted as $r^t = r_s^t + r_c^t$. Upon successful capture of the evader, the pursuers will receive a high mission success reward r_{win} , and the episode terminates. However, if any pursuit results in a collision, or exits the airspace, it will be deemed a mission failure, prompting an early termination of the episode and incurring a high penalty r_{fall} .

Global state: The global state at time t can be defined as $\mathbf{s}^t \in S$. The state $\mathbf{s}^t$ consists of the coordinates $\mathbf{p}^t$, yaw angle

ψ^t , velocity $\mathbf{v}^t$, and obstacle information $\mathcal{A}^t$ of all pursuers and the evader. Under the influence of joint action $\mathbf{u}_{jt}^t$, the environment undergoes updates, and the state $\mathbf{s}^t$ transitions according to the state transition function $\mathcal{P}(\mathbf{s}^{t+1}|\mathbf{s}^t, \mathbf{u}_{jt}^t)$, generating a new state $\mathbf{s}^{t+1}$. Agents can obtain new joint observations $\mathbf{o}_{jt}^{t+1}$ from the environment.

C. Algorithm Architecture

NAGC builds upon the classic Actor-Critic paradigm and employs a framework of centralized training with decentralized execution (CTDE). The structural layout and design concepts of Actor network and Critic network are delineated as follows:

1) *Normalizing Flow Actor Network*: In many classical continuous RL algorithms, the policy network typically adopts a method of generating a probability distribution to sample actions [18]. Compared to deterministic action generation methods (where the policy network directly outputs actions), this approach often offers enhanced flexibility and exploration. However, these classical algorithms are accustomed to generating Gaussian distributions. This simplicity can lead to limited modeling expressivity when tackling complex tasks. Directly utilizing a Gaussian distribution to approximate the optimal action distribution (which is unknown to us) is inherently imprecise, given the simplicity and unimodal nature of the Gaussian distribution. Although there are methods to generate more complex distributions [46], they face challenges in sampling actions with gradients, lose the advantageous properties of sampling from a simple distribution like a reparameterized Gaussian, and struggle to adapt to the maximum entropy RL training approach that requires the calculation of policy entropy.

To address the aforementioned challenges, we aim for the Actor network to acquire the capability of learning a complex distribution tailored to our mission, while retaining the advantages inherent in the facile reparameterization of sampled actions and entropy computation, akin to the Gaussian distribution. The normalizing flow elucidates the transformation of a probability density through a succession of invertible mappings. By iteratively applying the change of variables rule, the initial density seamlessly “flows” through this series of invertible mappings. Upon reaching the conclusion of this sequence, a new complex probability distribution is attained, hence earning the nomenclature of normalizing flow [47]. In NAGC, we introduce the normalizing flow (NF) method to enhance the Actor network.

The structure of the Actor network is illustrated in Fig. 4. Taking the agent’s observation as input, the information undergoes processing through a multilayer perceptron (MLP) by two fully connected (FC) layers, with rectified linear unit (ReLU) serving as the activation function. We use MLP_i^{base} to represent them. Subsequently, the output of MLP_i^{base} is subject to two independent FC layers, FC_i^μ and FC_i^σ . The output from FC_i^μ is utilized to generate the mean value μ_i of the Gaussian distribution, while the output from FC_i^σ is the logarithm of standard deviation $\log \sigma_i$ of the Gaussian distribution. The process can be expressed by the following formula:

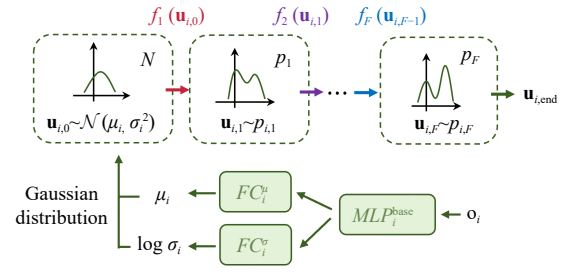


Fig. 4. A sampled action from a normalizing flow actor.

$$\begin{cases} \mu_i = FC_i^\mu(MLP_i^{\text{base}}(\mathbf{o}_i)) \\ \log \sigma_i = FC_i^\sigma(MLP_i^{\text{base}}(\mathbf{o}_i)). \end{cases} \quad (18)$$

By employing μ_i and $\log \sigma_i$, a Gaussian distribution $p_{i,0} = \mathcal{N}(\mu_i, \sigma_i^2)$ is generated. The initial action $\mathbf{u}_{i,0} \sim p_{i,0}$, is sampled from this distribution. To enable the gradient of the sampled action, reparameterized tricks are employed. Subsequently, the initial action, $\mathbf{u}_{i,0}$ will be input to the first flow layer f_1 of normalizing flows, and a new action, $\mathbf{u}_{i,1}$ will be generated by $\mathbf{u}_{i,1} = f_1(\mathbf{u}_{i,0})$.

After this transformation, the new action, $\mathbf{u}_{i,1}$ can be viewed as a sample from an updated distribution $p_{i,1}$, resulting from a modification in the original Gaussian distribution. In order to ascertain the specific transformation formula of flow f , we consider a classical invertible linear-time transformations formula, named Planar Flow [47], for its low-cost computation of the determinant. In our paper, flow f follows the equation:

$$f(\mathbf{u}) = \mathbf{u} + \mathcal{M}_z G(\mathcal{M}_w^T \mathbf{u} + \mathcal{M}_b) \quad (19)$$

where $\mathcal{M}_z \in \mathbb{R}^{D_u}$, $\mathcal{M}_w \in \mathbb{R}^{D_u}$ and $\mathcal{M}_b \in \mathbb{R}$ are free parameters, D_u is the dimension of $\mathbf{u}$. $G(\cdot)$ is a smooth element-wise non-linearity, with derivative $G'(\cdot)$. For this mapping we can compute the Jacobian determinant in $O(D_u)$ time (using the matrix determinant lemma)

$$\begin{cases} \varepsilon(\mathbf{u}) = G'(\mathcal{M}_w^T \mathbf{u} + \mathcal{M}_b) \mathcal{M}_w \\ \det \left| \frac{\partial f}{\partial \mathbf{u}} \right| = \left| \det(\mathbf{I} + \mathbf{u} \varepsilon(\mathbf{u})^T) \right| = |1 + \mathbf{u}^T \varepsilon(\mathbf{u})|. \end{cases} \quad (20)$$

Action $\mathbf{u}_{i,1}$ then serves as input for the subsequent flow layer f_2 , generating the subsequent action $\mathbf{u}_{i,2}$. Similarly, $\mathbf{u}_{i,2}$ can be seen as a sampled action from a new distribution $p_{i,2}$. This process is repeated in each flow layer, executing the transformation of (19). There is a total of F flows, so iterative process continues until all flow layers have been traversed, ultimately yielding the final action $\mathbf{u}_{i,\text{end}}$, as expressed by the following formula:

$$\mathbf{u}_{i,\text{end}} = f_F \circ f_{F-1} \circ \dots \circ f_2 \circ f_1(\mathbf{u}_{i,0}). \quad (21)$$

This form is a shorthand for $f_F(f_{F-1}(\dots f_1(x)))$. Notably, the parameters $\mathcal{M}_z$, $\mathcal{M}_w$ and $\mathcal{M}_b$ of each flow layer are different. To generate these parameters for all flow layers, we employ three FC layers FC_i^z , FC_i^w and FC_i^b , respectively. Each of networks is a single-layer network, and all take the observation $\mathbf{o}_i$ processed by MLP_i^{base} as input. The computation process is as follows:

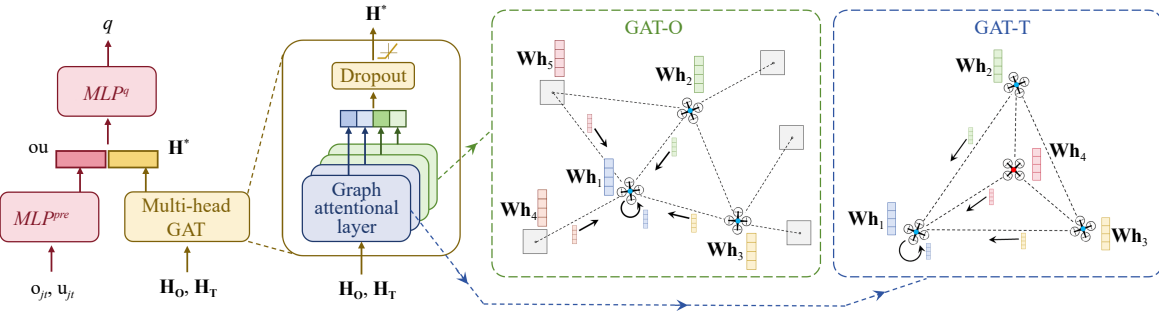


Fig. 5. The architecture of graph attention critic.

$$\begin{cases} [\mathcal{M}_{z,1}, \mathcal{M}_{z,2}, \dots, \mathcal{M}_{z,F}]_i = FC_i^z(MLP_i^{\text{base}}(o_i)) \\ [\mathcal{M}_{w,1}, \mathcal{M}_{w,2}, \dots, \mathcal{M}_{w,F}]_i = FC_i^w(MLP_i^{\text{base}}(o_i)) \\ [\mathcal{M}_{b,1}, \mathcal{M}_{b,2}, \dots, \mathcal{M}_{b,F}]_i = FC_i^b(MLP_i^{\text{base}}(o_i)). \end{cases} \quad (22)$$

By iteratively transforming F times, the action $\mathbf{u}_{i,0}$ sampled from the Gaussian distribution can be effectively transformed into one $\mathbf{u}_{i,\text{end}}$ sampled from a complex or multi-modal distribution. Moreover, FC_i^z , FC_i^w and FC_i^b also participate in the end-to-end updates of reinforcement learning, enabling the actor network to learn a complex distribution form suitable for the task.

2) *Graph Attention Critic Network*: In our problem, there are crucial spatial topological features between UAVs, or between UAVs and the environment that significantly impact task success. For instance, during the search phase, pursuit UAVs need to maintain a larger visual field and should consider their spatial relationship with surrounding obstacles. Similarly, during the pursuit phase, to effectively capture the evasion UAV, the pursuit UAVs need to focus on their spatial relationship with the evasion UAV. When we consider the UAVs and surrounding obstacles as nodes, the resulting topological graph formed by their connections becomes a key basis for evaluating the current situation. Therefore, in the Critic network of NAGC, we integrated an “obstacle-target” graph attention network. The “obstacle” graph attention network (GAT-O) is used to learn the graph features between the pursuit UAVs and obstacles, while the “target” graph attention network (GAT-T) is used to learn the graph features between the pursuit UAVs and the evasion UAV.

GAT-O and GAT-T will process different sets of graph structure data, with distinct inputs. For GAT-O, in addition to all pursuit UAVs, we consider the nearest κ obstacles for each pursuit UAV as nodes. Therefore, the input feature of all nodes in GAT-O can be denoted as $\mathbf{H}_O = [\mathbf{h}_{o_1}, \mathbf{h}_{o_2}, \dots, \mathbf{h}_{o_{N(\kappa+1)}}]$. Taking $\mathbf{h}_{o_i}$ as an example, if i represents a pursuit UAV, then $\mathbf{h}_{o_i}$ includes the UAV’s coordinates $\mathbf{p}_i$, yaw angle ψ_i . If i represents an obstacle, then $\mathbf{h}_{o_i}$ includes the coordinates of the obstacle. For GAT-T, the input feature of all nodes can be denoted as $\mathbf{H}_T = [\mathbf{h}_{t_1}, \mathbf{h}_{t_2}, \dots, \mathbf{h}_{t_N}, \mathbf{h}_{t_{N+1}}]$. Taking $\mathbf{h}_{t_i}$ as an example, it contains the coordinates $\mathbf{p}_i$, yaw angle ψ_i , velocity $\mathbf{v}_i$, and obstacle information λ_i of UAV i . To better understand the nodes considered by these two GATs, please refer to Fig. 5.

Since the computational mechanisms of GAT-O and GAT-T networks are consistent, we will use $\mathbf{h}$ to represent the input

node feature with dimension D_h , $\mathbf{h}'$ to represent the output node feature with dimension $D_{h'}$ when detailing the GAT computational process. To ensure sufficient expressive power for transforming input features into higher-level features, the graph attention network utilizes a shared weight matrix $\mathbf{W} \in \mathbb{R}^{D_{h'} \times D_h}$ for a learnable linear transformation. This matrix is applied to each node’s feature vector. Subsequently, self-attention is performed between nodes, and the *LeakyReLU* nonlinearity is employed as the activation function to obtain attention coefficients. Taking nodes i and j as an example, the calculation process for the attention coefficient ϵ_{ij} between these two points is as follows:

$$\epsilon_{ij} = \text{LeakyReLU}(\mathbf{a}[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]) \quad (23)$$

where $\mathbf{a} \in \mathbb{R}^{D_{h'}}$ represents a trainable weight vector, and $\parallel$ denotes the concatenation operation. The attention coefficient ϵ_{ij} signifies the significance of node j ’s feature concerning node i . For enhanced comparability across various nodes, these coefficients are normalized using the *softmax* function. For nodes i and j , the normalization is carried out using the following formula:

$$A_{ij} = \text{softmax}(\epsilon_{ij}) = \frac{\exp(\epsilon_{ij})}{\sum_{k \in \mathcal{B}_i} \exp(\epsilon_{ik})} \quad (24)$$

where $\mathcal{B}_i$ represents the set of adjacent nodes of node i . In GAT-O, all pursuit UAVs are adjacent to each other, and each pursuit UAV is also adjacent to the κ obstacle center points that are nearest to it. In GAT-T, all UAVs are adjacent to each other, as shown in Fig. 5. Once acquired, the normalized attention coefficients A are employed to calculate a linear combination of the features associated with them. This linear combination serves as the final output features for each node. To enhance the stability of the self-attention learning process, we extend this mechanism to incorporate multi-head attention. In particular, K independent attention heads perform the transformation. Subsequently, their features are concatenated, yielding the following output feature representation:

$$\mathbf{h}'_i = \parallel_{k=1}^K \sum_{j \in \mathcal{B}_i} A_{ij}^k \mathbf{W}^k \mathbf{h}_j \quad (25)$$

where $\parallel$ represents concatenation, A_{ij}^k denotes the normalized attention coefficients computed by the k -th attention mechanism, and $\mathbf{W}^k$ is the weight matrix for the k -th input linear transformation. The output features of all nodes in GAT-O are $\mathbf{H}_O' = [\mathbf{h}'_{o_1}, \mathbf{h}'_{o_2}, \dots, \mathbf{h}'_{o_{N(\kappa+1)}}]$, in GAT-T are $\mathbf{H}_T' =$

$[\mathbf{h}\mathbf{t}'_1, \mathbf{h}\mathbf{t}'_2, \dots, \mathbf{h}\mathbf{t}'_N, \mathbf{h}\mathbf{t}'_{N+1}]$. $\mathbf{H}\mathbf{O}'$ and $\mathbf{H}\mathbf{T}'$ will then be concatenated together and enter into a *Dropout* layer, which randomly sets a small number of values in the vector to zero to mitigate overfitting. Subsequently, using the exponential linear unit (*ELU*) as the activation function, the final graph feature $\mathbf{H}^*$ is obtained. This process can be expressed by the following formula:

$$\mathbf{H}^* = \text{ELU}(\text{Dropout}(\mathbf{H}\mathbf{O}' \parallel \mathbf{H}\mathbf{T}')). \quad (26)$$

To integrate the output of the GAT networks into the Critic network, we will first preprocess the joint observation $\mathbf{o}_{jt}$ and joint action $\mathbf{u}_{jt}$ using an MLP by the two-layer FC network, with *ReLU* as the activation function between the two layers. $\mathbf{o}_{jt}$ and $\mathbf{u}_{jt}$ will be concatenated together as the input to the network. We denote this part of the network as MLP^{pre} , and the computation process is described as follows:

$$\mathbf{ou} = MLP^{pre}(\mathbf{o}_{jt} \parallel \mathbf{u}_{jt}). \quad (27)$$

The vector $\mathbf{ou}$ will be concatenated with the output of GAT networks, and the resulting vector will be fed into a new MLP using *ReLU* as the activation function, while the final layer directly outputs the action value q , as shown in Fig. 5. We denote this part of the network as MLP^q . The computation process is as follows:

$$q = MLP^q(\mathbf{ou} \parallel \mathbf{H}^*). \quad (28)$$

Consequently, the Critic network not only utilizes the joint observations and joint action as the foundation for calculating the action value, more importantly, it also incorporates the graph features formed between UAVs and obstacles as a significant basis. This dual-source input enables a more precise assessment of the current situation, facilitating pursuers to develop more effective movement patterns that are conducive to the mission.

D. Training Algorithm

Our algorithm is based on the maximum entropy RL architecture. Compared to the traditional RL, it allows the policy network to maintain a certain level of entropy, ensuring that the algorithm retains a degree of exploration during training. Its objective is to learn a joint policy π_{jt} that maximizes the return and entropy [18], as follows:

$$r\pi_{jt}^* = \arg \max_{\pi_{jt}} \mathbb{E}_{(\mathbf{s}^t, \mathbf{u}_{jt}) \sim \mathcal{T}_{\pi_{jt}}} \left[\sum_{t=0}^{+\infty} \gamma^t (r(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) + \alpha \mathcal{H}(\pi_{jt}(\cdot | S^t = \mathbf{s}^t))) \right] \quad (29)$$

where $\mathcal{T}_{\pi_{jt}}$ denotes the trajectory distribution of the joint action of the state induced by joint policy π_{jt} , and $\mathcal{H}(\pi_{jt}(\cdot | S^t = \mathbf{s}^t))$ denotes the entropy of joint policy π_{jt} under state $\mathbf{s}^t$, it can be defined as

$$\mathcal{H}(\pi_{jt}(\cdot | S^t = \mathbf{s}^t)) \triangleq \mathbb{E}_{\mathbf{u}_{jt} \sim \pi_{jt}} \left[-\log \pi_{jt}(U^t = \mathbf{u}_{jt}^t | S^t = \mathbf{s}^t) \right] \quad (30)$$

where α is the temperature parameter that balances the importance of the reward and the entropy terms.

In the maximum entropy RL architecture, the state value function can be defined as

$$V(S^t = \mathbf{s}^t) = \mathbb{E}_{\mathbf{u}_{jt} \sim \pi_{jt}} \left[Q(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) - \alpha \log \pi_{jt}(U^t = \mathbf{u}_{jt}^t | S^t = \mathbf{s}^t) \right] \quad (31)$$

where the soft Q -function can be defined as follow:

$$Q(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) = r(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) + \gamma \mathbb{E}_{\mathbf{s}^{t+1} \sim \mathcal{T}} [V(S^{t+1} = \mathbf{s}^{t+1})]. \quad (32)$$

In our algorithm, $Q_\theta(\mathbf{s}^t, \mathbf{u}^t)$ is generated by the Q -value network, specifically, the Critic network with parameters θ . Hence, the parameters of the soft Q -function can be trained to minimize the following loss:

$$J_Q(\theta) = \mathbb{E}_{(\mathbf{s}^t, \mathbf{u}_{jt}) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) - r(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) - \gamma \mathbb{E}_{\mathbf{s}^{t+1} \sim \mathcal{T}} [V_{\bar{\theta}}(S^{t+1} = \mathbf{s}^{t+1})])^2 \right] \quad (33)$$

where $\mathcal{D}$ refers to the replay buffer, $\bar{\theta}$ represents the parameters of the target Q -function, which consists of a set of target networks mirroring the structures of the original networks. Target networks can effectively mitigate the overestimation problem [48]. They are updated through a soft-update approach, wherein the network parameters are updated by blending τ proportion of the parameters from the original networks and $1 - \tau$ proportion of their own parameters during every update step, that is

$$\bar{\theta} \leftarrow \tau \times \theta + (1 - \tau) \times \bar{\theta}. \quad (34)$$

Each agent's individual policy π is computed by its own Actor network with parameters ϕ . Therefore, each actor network can optimize its own parameters ϕ using the following equation:

$$J_\pi(\phi) = \mathbb{E}_{(\mathbf{s}^t, \mathbf{o}^t) \sim \mathcal{D}} \left[\mathbb{E}_{\mathbf{u}^t \sim \pi_\phi} \left[\alpha \log \pi_\phi(U^t = \mathbf{u}^t | O^t = \mathbf{o}^t) - Q_\theta(S^t = \mathbf{s}^t, U^t = \mathbf{u}_{jt}^t) \right] \right]. \quad (35)$$

The temperature parameter α can be adaptively adjusted during the training process by minimizing the following loss:

$$J(\alpha) = \mathbb{E}_{\mathbf{s}^t \sim \mathcal{D}} \left[-\alpha \log \pi_{jt}(U^t = \mathbf{u}_{jt}^t | S^t = \mathbf{s}^t) - \alpha \bar{\mathcal{H}} \right] \quad (36)$$

where $\bar{\mathcal{H}}$ represents the target entropy of the joint policy. It is often pre-set to a fixed value. Therefore, the critic network with a learning rate lr_Q , the actor network with a learning rate lr_π , and the entropy with a learning rate lr_α will be updated at each training step according to (33), (35), and (36), respectively.

The pseudo-code of NAGC is provided in Algorithm 1. The computational complexity analysis of NAGC is shown in S4 of Supplementary Material¹.

V. EXPERIMENT

In this section, we investigate performance of the proposed NAGC for MUV-PE problem. We adopt four state-of-the-art MARL algorithms as baselines. Multiple representative metrics are employed to evaluate the performance of different

algorithms. Additionally, we showcase the generalization capability of NAGC across multiple scenarios and verify the effectiveness of each module of NAGC in ablation study. Finally, we analyze the behaviors derived from NAGC to give insights into pursuit-evasion.

Algorithm 1 NAGC Algorithm

- 1: Initialize policy network $\pi_{\phi_1}, \pi_{\phi_2}, \dots, \pi_{\phi_N}$, and Q -value network $Q_{\theta_1}, Q_{\theta_2}$
 - 2: Initialize target Q -value network $Q_{\bar{\theta}_1}, Q_{\bar{\theta}_2}$, and $\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$
 - 3: Initialize an empty replay buffer $\mathcal{D} \leftarrow \emptyset$ with size $\mathcal{Z}$
 - 4: **for** $episode \leftarrow 1$ **to** EP **do**
 - 5: **for** $t \leftarrow 1$ **to** $T_{\max}$ **do**
 - 6: Get global state $\mathbf{s}^t$ and joint observation $\mathbf{o}_{jt}^t$ of environment
 - 7: **for** $i \leftarrow 1$ **to** N **do**
 - 8: Get individual observation $\mathbf{o}_i^t$ from $\mathbf{o}_{jt}^t$
 - 9: Select action $\mathbf{u}_i^t$ by $\pi_{\phi_i}(\mathbf{u}_i^t | \mathbf{o}_i^t)$
 - 10: **end for**
 - 11: Execute the joint action $\mathbf{u}_{jt}^t$
 - 12: Receive reward r^t and get global state $\mathbf{s}^{t+1}$ and joint observation $\mathbf{o}_{jt}^{t+1}$ of the updated environment
 - 13: Store transition $(\mathbf{o}_{jt}^t, \mathbf{s}^t, \mathbf{u}_{jt}^t, r^t, \mathbf{o}_{jt}^{t+1}, \mathbf{s}^{t+1})$ in $\mathcal{D}$
 - 14: Sample a random batch of transitions from $\mathcal{D}$
 - 15: Update Q -value network by minimizing the loss in (33), $\theta_k \leftarrow \theta_k - lr_Q \hat{\nabla}_{\theta_k} J_Q(\theta_k)$ for $k \in \{1, 2\}$
 - 16: Update each policy network by minimizing the loss in (35), $\phi_i \leftarrow \phi_i - lr_\pi \hat{\nabla}_{\phi_i} J_\pi(\phi_i)$ for $i \in \{1, \dots, N\}$
 - 17: Update the temperature parameter by minimizing the loss in (36), $\alpha \leftarrow \alpha - lr_\alpha \hat{\nabla}_\alpha J(\alpha)$
 - 18: Update the target network $\bar{\theta}_k \leftarrow \tau \times \theta_k + (1 - \tau) \times \bar{\theta}_k$ for $k \in \{1, 2\}$
 - 19: **end for**
 - 20: **end for**
-

A. Simulation Environment Settings

1) *Airspace Environment*: We employed unreal engine² (UE) to construct the scene for conducting simulation experiments, alongside AirSim³ for executing UAV flight control. As shown in Fig. 6, we defined a square airspace with dimensions of 1 km by 1 km, and all UAVs are restricted to operate within this area. The center point of the square airspace is located at [0 m, 0 m]. Within the airspace, there are 16 buildings of the same size, each with a height above the flight altitude of the UAVs, thus obstructing the UAVs' vision and movement. Each building has dimensions of 50 m by 50 m, with their sides parallel to the x-axis and y-axis of the airspace plane.

2) *UAVs*: The initial coordinates of the three pursuit UAVs are fixed at [0 m, 350 m], [-250 m, -250 m], and [-250 m, 250 m]. The initial coordinates of the evasion UAV can appear in any non-building area within the airspace, and these coordinates are randomly generated at the beginning of each

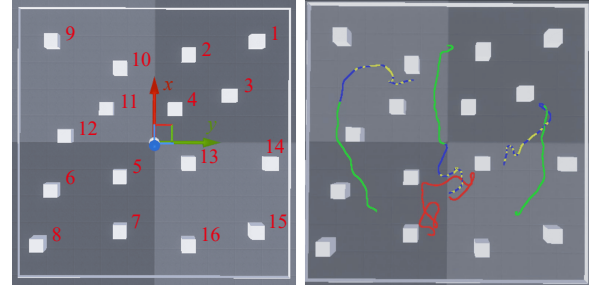


Fig. 6. Multi-building bounded urban airspace scenario. The center point coordinates of each building can be found in S5 of Supplementary Material by its ID. We use lines of various colors to depict the trajectories of UAVs in different phases. Green represents the pursuit UAVs are in the search phase, blue represents the pursuit UAVs are in the pursuit phase, and yellow represents the pursuit UAVs are in the reacquisition phase (approach). The red specifically represents the trajectory of evasion UAV.

episode. We set the various parameters of the UAVs based on realistic UAV specifications. All parameters related to the UAVs can be found in Table I, which are already feasible in reality [2], [3], [6]. All UAVs are large quadrotor drones of the model Parrot AR.Drone, and the flight control used is AirSim's default *Simple_Flight*. We send action commands to all UAVs simultaneously every second in the simulator, with the maximum duration of the task set to 180 s. The task can end prematurely if a pursuit UAV leaves the designated airspace, collides, or captures the evasion UAV.

TABLE I
PARAMETERS OF UAV

Parameters	Values
Size	1 m × 1 m
Angular velocity ω	$-\pi/3 \sim \pi/3$ rad/s
Velocity of evader v_e	12 m/s
Velocity of pursuers v_i	9–11 m/s
Acceleration of pursuers a_i	$-1 \sim 1$ m/s ²
Perception distance of visual field L	200 m
Viewing angle of visual field ϕ	120°
Capture radius δ_{cap}	15 m
Minimum safe distance δ_s	1 m
Maximum length of radar ray $\lambda_{\max}$	100 m
Obstacle warning distance δ_w	25 m

B. Evasion Strategy

In this experiment, three distinct evasion strategies for the evader were established:

1) *Random Move Strategy*: The evader maneuvers at maximum speed, while ensuring that the yaw angular velocity at each moment adheres to a random value constrained by the UAV dynamics equation.

2) *Repulsive Force Strategy*: Under this strategy, the evader operates at maximum speed and seeks escape by navigating towards the direction of the resultant repulsive force. The calculation process of repulsive force can be seen in S6 of Sup-

² More details at <https://www.unrealengine.com>

³ It can be download at <https://microsoft.github.io/AirSim/>

TABLE II
THE PARAMETERS OF ALGORITHM AND REWARD FUNCTION

Parameters	Value	Parameters	Value
Training steps	5×10^5	Learning rate of actor	1×10^{-4}
Batch size	256	Learning rate of critic	1×10^{-3}
Discount factor	0.95	Learning rate of entropy	1×10^{-3}
Optimizer	Adam	Replay buffer size	5×10^5
Attention heads	2 and 2	Soft-update coefficient	0.01
Flow number	4	Dropout probability	0.01
Hidden dimension	64	LeakyReLU coefficient	0.2
Constant C_1	1.5×10^5	Constant C_2	2×10^3
Constant C_3	1×10^4	Constant C_4	100
Constant C_5	0.25	Constant C_6	0.02
Constant C_7	2.5×10^3	Constant C_8	-0.1

plementary Material¹.

3) *Reinforcement Learning Strategy*: Under this strategy, The evader will use a pre-trained neural network to decide its movements, which was trained by the classic single-agent reinforcement learning algorithm: Deterministic deep policy gradient (DDPG) [22]. The detailed Markov decision process is included in S6 of Supplementary Material¹.

Among the three evasion strategies, in order to entirely avoid collisions, obstacle avoidance rule was established for the evader. If the evader approaches within 25 meters of an obstacle, it will maneuver to deviate from the obstacle's direction as much as possible. It will then maintain a safe distance of over 25 meters from the obstacle to ensure avoidance of collision. This rule is also used for preventing evader from leaving the designated airspace.

C. Comparative Experiment

The parameter settings for our algorithm and the reward function are shown in the Table II. Apart from the NAGC algorithm proposed in this paper, we have selected four state-of-the-art MARL methods (available for continuous control): HASAC [49], FACMAC [20], HAPPO [50], and MAPPO [19] for comparison. This experiment was conducted using a device equipped with an i7-13700KF processor, 4070 GPU, and 32 GB RAM. Before the formal training of the algorithm begins, we perform 1×10^4 steps of random action selection and retain the experiences sampled during this random exploration, to enhance the algorithm's initial environmental exploration. Under the three evasion strategies, we evaluate the network after every 1×10^4 steps during training. Each evaluation contains 50 episodes. To compare the performances of different algorithms, we not only considered the accumulative reward, but also designed five evaluation metrics tailored to the problem's characteristics:

1) *Success Capture Rate (SCR)*: During each evaluation, we calculate the proportion of episodes in which the evader was successfully captured. The proportion is the value of the SCR metric. This is a positive metric.

2) *Mission Time*: During each evaluation, we calculate the average time of completing the mission within an episode. The average mission time is the value of the mission time

metric. Note that if any pursuer leaves the airspace or collides, the episode will terminate immediately. For these failed episodes, the mission time is calculated as the maximum mission time. Therefore, shorter times indicate higher efficiency in mission completion.

3) *Collision-Departure Rate (CDR)*: During each evaluation, we calculate the proportion of episodes in which pursuers collide or leave the airspace. The proportion of these episodes is the value of the CDR metric. This is a negative metric.

4) *Search Time*: During each evaluation, we calculate the average time for pursuers to initially search the evader. The average search time is the value of the Search Time metric. Search time starts accumulating from the beginning of an episode and ends when any pursuer detects the evader. If any pursuer leaves the airspace or collides before the evader is detected, the search time for that episode is recorded as the maximum mission time. Shorter search times indicate a more effective search strategy.

5) *Lost Time*: During each evaluation, we record the average total duration the pursuers lose sight of the evader within an episode. The average lost time is the value of the Lost Time metric. The lost time for an episode begins when the evader leaves the visual field of all pursuers and ends when it is rediscovered. If the evader is lost again, the lost time continues to accumulate until the evader is rediscovered or the episode ends. To better evaluate the pursuers' strategy in handling target loss, we only count the lost time in episodes where the target was lost. Shorter duration indicates an improved ability of the pursuers to maintain visual contact with the target.

The results for each evaluation metric from repeated experiments are presented in the table from S7 of Supplementary Material¹. Additionally, we focused on the two crucial metrics, ie. reward, and SCR, and generated corresponding learning curves for different algorithms, as illustrated in Figs. 7 and 8. In these figures, we can clearly see the performance of each algorithm during the training process.

From the perspective of reward acquisition, NAGC achieves the highest reward under any evasion strategy. Under the random move evasion strategy, the reward differences among all

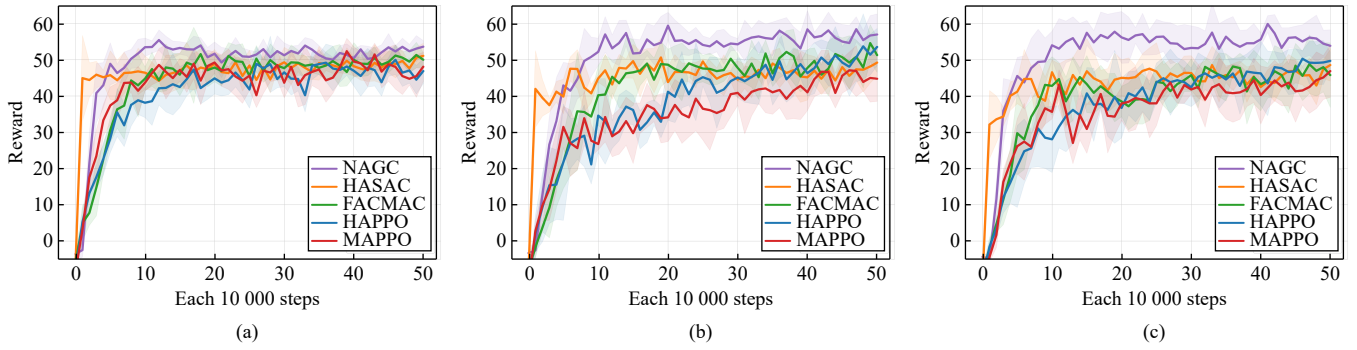


Fig. 7. Learning curves of rewards under three evasion strategy, for comparison of NAGC and other MARL algorithms. Random move, repulsive force, and reinforcement learning evasion strategies are represented from (a) to (c).

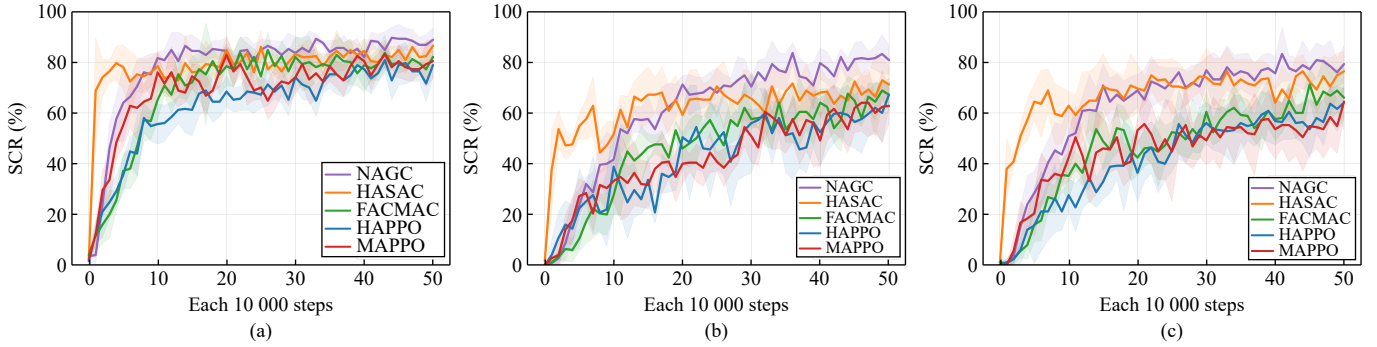


Fig. 8. Learning curves of SCR under three evasion strategy, for comparison of NAGC and other MARL algorithms. Random move, repulsive force, and reinforcement learning evasion strategies are represented from (a) to (c).

algorithms are relatively small, as shown in Fig. 7(a). However, under the other two evasion strategies, NAGC's stronger reward optimization capability is more evident. In the repulse force evasion strategy, NAGC surpasses other algorithms by an average of 3.45 to 12.19, as shown in Fig. 7(b). Under the reinforcement learning evasion strategy, NAGC's rewards are higher than those of other algorithms, exceeding HAPPO by 8.4%, as shown in Fig. 7(c). This analysis demonstrates NAGC's promising performance in optimizing rewards.

From the perspective of evaluation metrics, NAGC achieves the highest SCR while maintaining a very low CDR. Under the relatively easy random move evasion strategy, NAGC achieves an SCR of 88.8%, as shown in Fig. 8(a), with a very low CDR of only 1.6%. In comparison, FACMAC, HAPPO, and MAPPO can only reach an SCR of around 80%, with CDR exceeding 5%. A clear comparison of CDR is shown in Fig. 9. Under the repulse force evasion strategy, other algorithms struggle to maintain an SCD of above 75%, but NAGC reaches an SCD of 80.8%, as shown in Fig. 8(b), and NAGC also has the lowest CDR, while HASAC and FACMAC have CDR values exceeding 10%, as shown in Fig. 9. Under the reinforcement learning evasion strategy, NAGC's SCR slightly decreases but still remains the highest, surpassing HASAC by 3.7%, as shown in Fig. 8(c), though its CDR is slightly worse than MAPPO by 2.4%. This analysis shows that NAGC effectively achieves high task success rates with fewer faults. In S10 of Supplementary Material¹, we depict the trajectories of two kinds of pursuers capturing the evader under NAGC algorithm.

Faster search and capture of evasion UAV is also a key

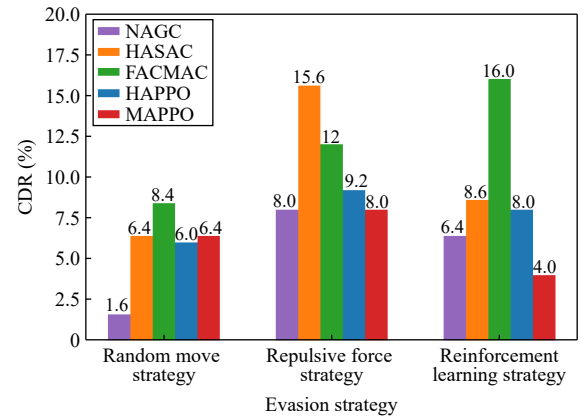


Fig. 9. Comparison of CDR performance of different algorithms under three evasion strategies.

focus of this problem. In terms of total mission time, NAGC consistently maintains the shortest duration under various evasion strategies, as shown in Fig. 10, where the total length of the bars represents the average mission time for each algorithm. HASAC lags slightly behind NAGC, while HAPPO and MAPPO have higher mission time. For example, under the three evasion strategies, MAPPO's mission time increases by 22%, 30.4%, and 33.2% respectively than NAGC. NAGC also performs well in search time, quickly locating evaders under both the repulse force and reinforcement learning evasion strategies, as shown by the green bars in Fig. 10. Under the random move evasion strategy, it is only 2.23 s slower than HASAC. Regarding lost time, NAGC has a significant

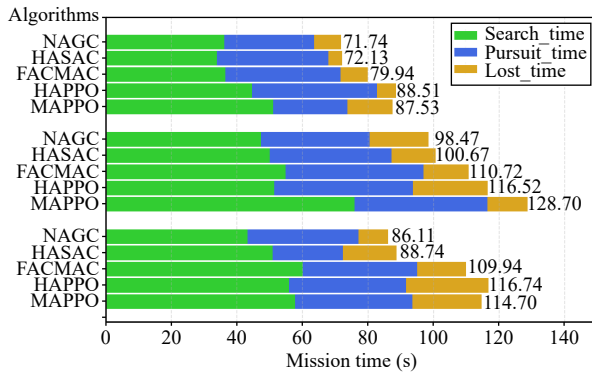


Fig. 10. A comparison chart of the time consumption for various states of different algorithms during the mission. The total length of the bar represents the mission time. The green bar indicates the search time, which is the time taken for the initial search for the evader. The blue bar represents the pursuit time, which is the time when the evader is within the pursuer's visual field. The yellow bar represents the lost time, which is the accumulated time that the pursuers have previously detected the evader but it has lost from the visual field. Since the lost time appears intermittently, for convenience, we have placed them all at the end of the bar. In the chart, from top to bottom, the evasion strategies are random move evasion strategy, repulse force evasion strategy, and reinforcement learning evasion strategy.

advantage under the reinforcement learning evasion strategy, keeping the average lost time below 10 s, while other algorithms exceed 10 s on average, as shown by the yellow bars in Fig. 10. This analysis shows that NAGC can complete the mission quickly and efficiently search and capture evader. Furthermore, we compared the computational load of different algorithms, the results and analysis are presented in S4 of Supplementary Material¹.

D. Generalization Experiment

To validate the adaptability of our algorithm in complex environments with different obstacle layouts, localization noise, and varying scales of agents, we designed six simulation environments with distinct characteristics. In these environments, the evaders employ a reinforcement learning evasion strategy. Additionally, we evaluated the algorithm's generalization performance in two new tasks. Note that we used the trained neural network with the highest success rate for each algorithm from the comparison experiments to test generalization.

1) *Environments With Different and Unfamiliar Obstacle Layouts*: We created two environments with different obstacle configurations: One with a higher obstacle density (Scenario 1) and the other with two new building types (Scenario 2). In both scenarios, the positions of the buildings are randomized compared to the training environment, as shown in Figs. 11(a) and 11(b).

2) *Environments With Localization Noise*: We designed an environment where the pursuers have localization errors (Scenario 3). The localization error of each agent follows a normal distribution with a mean of 0 and a standard deviation of 10 meters. Additionally, we created an environment where the pursuers receive noisy position information about the evaders (Scenario 4). The noise follows a normal distribution with a

mean of 0 and a standard deviation of $0.1d_{\min}$, where $d_{\min}$ is the shortest distance between the evader and the pursuers which detect the evader. If the evader is outside all pursuers' visual field, the pursuers do not obtain the evader's position. Please note that Scenarios 3 and 4 have the same building layout as the training scenario in Fig. 6. Therefore, they are not repeated in Fig. 11.

3) *Environments With Different Scales of Agents*: We set up two environments with larger scales of agents, increasing the number of agents from 3 to 5 (Scenario 5) and 8 (Scenario 6). The initial positions of the agents are shown in Figs. 11(c) and 11(d). To maintain the neural network's input dimensions, we assumed that each agent communicates with the two nearest agents. Additionally, we allowed the number of UAVs involved in collisions to be no more than 1 in Scenario 5 and 2 in Scenario 8.

4) *New Tasks*: We introduced two new tasks to test the algorithm's generalization capabilities. The first task is searching for multiple static targets (Scenario 7). In this scenario, we randomized the positions of obstacles and randomly generated five static targets in each episode. A target is considered found if it appears within the visual field of any UAV. The multi-UAV system needs to locate as many targets as possible within 180 s, as shown in Fig. 11(e). We used the average number of targets found and the average search time spent per target as evaluation metrics, referred to as Metric 1 and Metric 2 here respectively. The second task is a multi-UAV navigation mission (Scenario 8). We disrupt the positions of environmental obstacles and randomly generate target points for each UAV within the airspace for each episode. Each UAV needs to navigate within the designated airspace, avoid obstacles, and reach its assigned target point as quickly as possible. If the three UAVs all safely reach their respective target points within 180 s, the mission is considered successful, as shown in Fig. 11(f). For this task, we have set the mission success rate and the average mission time (i.e., the average time taken to complete the mission) as evaluation metrics.

Each scenario undergoes repeated experiments, and the average results of six evaluation metrics in Scenario 1 to Scenario 8 are presented in tables from S8 of Supplementary Material¹. It can be seen that altering the obstacle layout in the environment does not significantly impact the performance of NAGC. Notably, in Scenario 2, NAGC also demonstrated a very high success rate and maintained a collision rate below 10% in an unfamiliar environment. The results from Scenario 3 and Scenario 4 indicate that NAGC exhibits good stability in noisy environments, ensuring a success rate of over 75%. In contrast, HAPPO and MAPPO struggle to generalize to scenarios with localization noise, with a noticeable drop in success rates. For example, in Scenario 4, where noise exists in evader localization, the SCR of HAPPO dropped to 48.8%, a 31.1% decrease compared to the original training result. From the performances in Scenarios 5 and 6, it is evident that increasing the number of UAVs significantly boosts NAGC's SCR, reaching an impressive 95.6% when the number of UAVs is increased to 8. It is noted that, increasing the number of UAVs in other algorithms does not necessarily improve the success rate, like HASAC and HAPPO, they struggle to

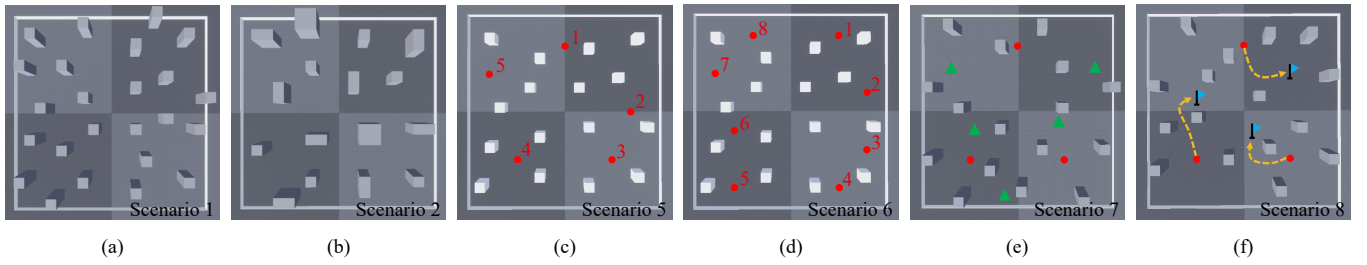


Fig. 11. Scenarios of generalization experiment. (a) and (b) represent environments with two different obstacle layouts. (a) shows an environment with denser obstacles, while (b) shows an environment with two new types of obstacles. (c) and (d) represent environments with larger UAV fleets. (c) marks the initial positions of 5 UAVs, while (d) marks the initial positions of 8 UAVs. (e) and (f) illustrate two new tasks. In (e), we use red circles to mark the initial positions of the UAVs, and green triangles to indicate the target points that need to be searched. Note that the positions of target points are completely random. In (f), we use red circles to mark the initial positions of the UAVs and blue flags to indicate the target points the UAVs need to reach. Note that the positions of target points are completely random.

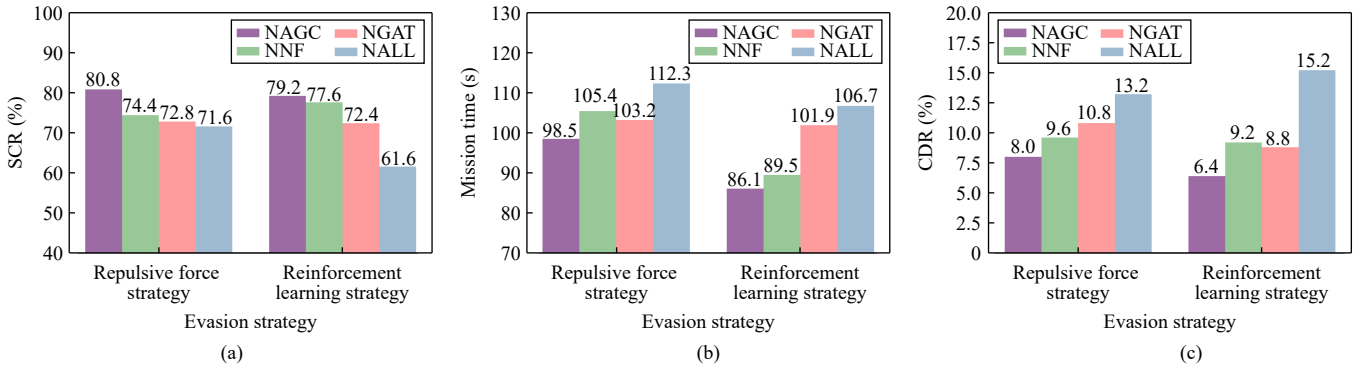


Fig. 12. Ablation study of the NAGC algorithm: Comparison of three metrics under two evasion strategies. (a) shows the results of SCR metric, and (b) shows the results of mission time metric, and (c) shows the results of CDR metric.

leverage the advantages brought by additional UAVs, failing to achieve strategic and highly coordinated capture. Instead, the inclusion of more UAVs often leads to chaotic actions. In Scenario 7, NAGC can quickly locate as many targets as possible when facing new search task. It is worth noting that we changed the obstacle layout in Scenario 7, and NAGC still maintained its advantage in this unfamiliar environment. In Scenario 8, we also altered the obstacle layout. Despite being trained for cooperative pursuit-evasion tasks, NAGC achieved over 80% success in multi-target point navigation task, with each agent navigating quickly to its target point while avoiding obstacles. In contrast, HAPPO struggled to generalize to the new navigation task, achieving only a 42% success rate. In summary, our experiments confirm that the NAGC algorithm can effectively adapt to complex environments with multiple obstacles, localization noise, and increased number of UAVs. Additionally, NAGC demonstrates promising performance in new tasks such as multi-target search and multi-target point navigation. Furthermore, we also test the generalization performance of NAGC in maze, communication loss scenario and scenario where evader moves close to the building, as shown in S10 of Supplementary Material¹.

E. Ablation Study

We conducted further ablation experiments on the NAGC algorithm, focusing primarily on three types of ablations. The first is the ablation of the NF, where the policy network directly generates actions by sampling from a Gaussian distribution.

In this paper, we denote this as NNF. The second is the ablation of the GAT, where the Critic network directly processes joint actions and joint observations using an MLP to generate action values. We denote this as NGAT. The third is the ablation of both NF and GAT, where the policy network samples actions using a Gaussian distribution and the Critic network directly processes joint actions and joint observations using an MLP to generate action values. In this case, it is equivalent to the multi-agent SAC algorithm, and we denote this as NALL.

Due to the fact that all algorithms can exhibit high performance under the random move evasion strategy, making the distinction less clear. In this experiment, we only consider that the evader employs the repulsive force strategy and reinforcement learning strategy. For the evaluation metrics, we selected three critical indicators: SCR, Mission time, and CDR. The results of these three metrics under two different evasion strategies are presented in Fig. 12.

As shown in Fig. 12(a), the GAT module and NF module both significantly improve the SCR. The reduction in SCR caused by the elimination of the GAT module is more pronounced than the elimination of the NF module. This is because the GAT module can effectively enhance the spatial coordination between multiple pursuers, facilitating the capture of the evader. Comparing NAGC and NNF, the NF-generated more complex distribution brings 6.4% and 1.6% increases respectively in SCR compared to directly sampling actions from a Gaussian distribution. Fig. 12(b) shows that

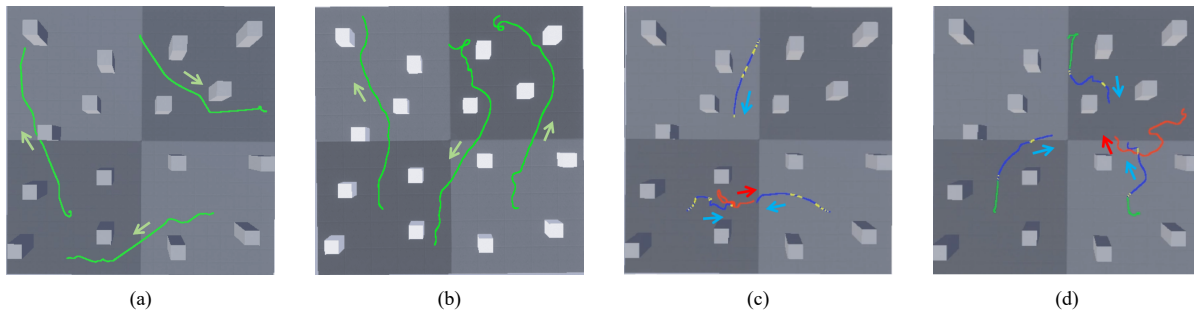


Fig. 13. Excellent collaborative behaviors of pursuers. (a) Spiral search; (b) Interleaving search; (c) Arrive preemptively; (d) Form an encircling movement pattern. The meanings of the different colored lines have already been explained in Fig. 6.

both the NF and GAT modules significantly reduce mission time. Under the repulsion evasion strategy, the NF module provides a 2.2 s benefit over the GAT module. In contrast, under the reinforcement learning evasion strategy, the GAT module offers a 12.4 s benefit over the NF module. When both modules are present, the lowest mission time is achieved under both evasion strategies. From Fig. 12(c), it is evident that both the NF and GAT modules significantly reduce the CDR, particularly under the reinforcement learning evasion strategy, where adding either module results in an approximate 40% reduction in the CDR compared to NALL. Under the repulsion evasion strategy, the GAT module results in a 3.6% reduction in CDR, while the NF module results in a 2.4% reduction. In summary, regarding the three critical metrics mentioned above, the benefits provided by the GAT and NF modules vary across different evasion strategies, making it difficult to determine definitively which module offers superior benefits. More importantly, removing either module leads to a significant performance decline in NAGC. When both the NF and GAT modules are present, they do not conflict but instead complement each other, enabling NAGC to achieve promising performance. Furthermore, we compared the influence of GAT on motion trajectories, the analysis is included in S9 of Supplementary Material¹.

F. Behavior Analysis

After training, the pursuers exhibit several excellent collaborative behaviors that notably enhance the success rate of captures. In the search phase, three pursuers typically adopt a spiral or interleaving search strategy, effectively hastening the discovery of the evader, as depicted in Figs. 13(a) and 13(b). In the pursuit phase, the pursuers manage to arrive preemptively at the potential path points of the evader, as shown in Fig. 13(c). Additionally, well-trained pursuers demonstrate the ability to sustain visual contact with the evader for a time as long as possible upon detection, reducing the duration that evader remains out of sight. Simultaneously, the pursuers tend to form an encircling movement pattern, gradually narrowing the encirclement, leading to a considerable increase in the success rate of capture, as shown in Fig. 13(d).

VI. CONCLUSION AND FUTURE WORK

In this paper, we introduce the MUV-PE problem and propose an MARL method for its efficient solution. The MUV-PE problem involves the pursuit UAVs utilizing a limited

visual field to locate and capture the evasion UAV. This problem aligns closely with real-life scenarios where UAVs equipped with sensors are employed to detect and capture unauthorized UAVs. Additionally, it has potential applications in complex environments, such as search and rescue missions for missing people, monitoring marine life with multiple underwater vehicles, and other meaningful scenarios. To address this problem, we enhance the MARL algorithm by incorporating normalizing flow and graph attention networks. The NAGC exhibits a more intricate capability for expressing policies and learning graph features, making it particularly suitable for the MUV-PE problem. Extensive experimentation reveals that the NAGC algorithm consistently outperforms state-of-the-art MARL algorithms across various metrics. Moreover, NAGC demonstrates well generalization to new tasks, larger-scale agent systems, and more complex obstacle scenarios. In future work, we will consider more realistic limitations for the MUV-PE problem, such as communication constraints among multiple UAVs, localization errors of UAVs, and partial UAV damage among swarm. We will also expand the MUV-PE problem into three dimension space. Furthermore, the NAGC algorithm can be expanded to other complex scenarios, such as multi-UAV formation control and multi-agent collaborative navigation in cluttered environments.

REFERENCES

- [1] Z. Zuo, C. Liu, Q.-L. Han, and J. Song, "Unmanned aerial vehicles: Control methods and future challenges," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 4, pp. 601–614, 2022.
- [2] C. de Souza Junior, "Hunter drones: Drones cooperation for tracking an intruder drone," Ph.D. dissertation, University of Technology of Compiègne, Compiègne, France, 2021.
- [3] T. Lefebvre and T. Dubot, "Conceptual design study of an anti-drone drone," in *Proc. 16th AIAA Aviation Technology, Integration, and Operations Conf.*, 2016, p. 3449.
- [4] T. H. Chung, G. A. Hollinger, and V. Isler, "Search and pursuit-evasion in mobile robotics: A survey," *Autonomous Robots*, vol. 31, pp. 299–316, 2011.
- [5] R. Zhang, Q. Zong, X. Zhang, L. Dou, and B. Tian, "Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning," *IEEE Trans. Neural Networks and Learning Systems*, vol. 34, no. 10, pp. 7900–7909, 2022.
- [6] Y.-C. Lai and T.-Y. Lin, "Vision-based mid-air object detection and avoidance approach for small unmanned aerial vehicles with deep learning and risk assessment," *Remote Sensing*, vol. 16, no. 5, p. 756, 2024.

- [7] C. De Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulić, "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robotics and Automation Lett.*, vol. 6, no. 3, pp. 4552–4559, 2021.
- [8] T. Olsen, A. M. Tumlin, N. M. Stiffler, and J. M. O'Kane, "A visibility roadmap sampling approach for a multi-robot visibility-based pursuit-evasion problem," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2021, pp. 7957–7964.
- [9] I. E. Weintraub, M. Pachter, and E. Garcia, "An introduction to pursuit-evasion differential games," in *Proc. American Control Conf.*, 2020, pp. 1049–1066.
- [10] N. Chen, L. Li, and W. Mao, "Equilibrium strategy of the pursuit-evasion game in three-dimensional space," *IEEE/CAA J. Autom. Sinica*, vol. 11, no. 2, pp. 1–13, 2024.
- [11] J. Chen, W. Zha, Z. Peng, and D. Gu, "Multi-player pursuit-evasion games with one superior evader," *Automatica*, vol. 71, pp. 24–32, 2016.
- [12] X. Qu, W. Gan, D. Song, and L. Zhou, "Pursuit-evasion game strategy of USV based on deep reinforcement learning in complex multi-obstacle environment," *Ocean Engineering*, vol. 273, p. 114016, 2023.
- [13] B. P. L. Lau, B. J. Y. Ong, L. K. Y. Loh, R. Liu, C. Yuen, G. S. Soh, and U.-X. Tan, "Multi-AGV's temporal memory-based RRT exploration in unknown environment," *IEEE Robotics and Automation Lett.*, vol. 7, no. 4, pp. 9256–9263, 2022.
- [14] Y. Wang, L. Dong, and C. Sun, "Cooperative control for multi-player pursuit-evasion games with reinforcement learning," *Neurocomputing*, vol. 412, pp. 101–114, 2020.
- [15] S. F. Desouky and H. M. Schwartz, "Self-learning fuzzy logic controllers for pursuit-evasion differential games," *Robotics and Autonomous Systems*, vol. 59, no. 1, pp. 22–33, 2011.
- [16] X. Fu, H. Wang, and Z. Xu, "Cooperative pursuit strategy for multiUAVs based on DE-MADDPG algorithm," *Acta Aeronautica Astronautica Sinica*, vol. 43, no. 5, pp. 325311–325311, 2022.
- [17] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in *Handbook of Reinforcement Learning and Control*, Urbana, USA: Springer, 2021, vol. 325, pp. 321–384.
- [18] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Offpolicy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Machine Learning*, 2018, pp. 1861–1870.
- [19] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems*, 2022, pp. 24611–24624.
- [20] B. Peng, T. Rashid, C. Schroeder de Witt, P.-A. Kamienny, P. Torr, W. Böhmer, and S. Whiteson, "FACMAC: Factored multi-agent centralised policy gradients," in *Proc. Advances in Neural Information Processing Systems*, 2021, pp. 12208–12221.
- [21] J. Ackermann, V. Gabler, T. Osa, and M. Sugiyama, "Reducing overestimation bias in multi-agent domains using double centralized critics," arXiv preprint arXiv: 1910.01465, 2019.
- [22] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," arXiv preprint arXiv: 1509.02971, 2015.
- [23] R. Isaacs, *Differential Games I: Introduction*. Santa Monica, USA: Rand Corporation, 1954.
- [24] T. D. Parsons, "Pursuit-evasion in a graph," in *Proc. Theory and Applications of Graphs: Proceedings*, 2006, pp. 426–441.
- [25] D. W. Oyler, P. T. Kabamba, and A. R. Girard, "Pursuit-evasion games in the presence of obstacles," *Automatica*, vol. 65, pp. 1–11, 2016.
- [26] I. Suzuki and M. Yamashita, "Searching for a mobile intruder in a polygonal region," *SIAM J. Computing*, vol. 21, no. 5, pp. 863–888, 1992.
- [27] B. P. Gerkey, S. Thrun, and G. Gordon, "Visibility-based pursuit-evasion with limited field of view," *The Int. J. Robotics Research*, vol. 25, no. 4, pp. 299–315, 2006.
- [28] B. Tovar and S. M. LaValle, "Visibility-based pursuit-evasion with bounded speed," *The Int. J. Robotics Research*, vol. 27, no. 11–12, pp. 1350–1360, 1350.
- [29] S. Sachs, S. M. LaValle, and S. Rajko, "Visibility-based pursuit-evasion in an unknown planar environment," *The Int. J. Robotics Research*, vol. 23, no. 1, pp. 3–26, 2004.
- [30] A. Dumitrescu, H. Kok, I. Suzuki, and P. Żyliński, "Vision-based pursuit-evasion in a grid," *SIAM J. Discrete Mathematics*, vol. 24, no. 3, pp. 1177–1204, 2010.
- [31] X. Liang, B. Zhou, L. Jiang, G. Meng, and Y. Xiu, "Collaborative pursuit-evasion game of multi-UAVs based on Apollonius circle in the environment with obstacle," *Connection Science*, vol. 35, no. 1, p. 2168253, 2023.
- [32] E. Lozano, U. Ruiz, I. Becerra, and R. Murrieta-Cid, "Surveillance and collision-free tracking of an aggressive evader with an actuated sensor pursuer," *IEEE Robotics and Automation Lett.*, vol. 7, no. 3, pp. 6854–6861, 2022.
- [33] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. S. Kumar, S. Koenig, and H. Choset, "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2378–2385, 2019.
- [34] Z. Feng, M. Huang, D. Wu, E. Q. Wu, and C. Yuen, "Multi-agent reinforcement learning with policy clipping and average evaluation for uav-assisted communication Markov game," *IEEE Trans. Intelligent Transportation Systems*, vol. 24, no. 12, pp. 14281–14293, Dec. 2023.
- [35] K. Xue, J. Xu, L. Yuan, M. Li, C. Qian, Z. Zhang, and Y. Yu, "Multi-agent dynamic algorithm configuration," in *Proc. Advances in Neural Information Processing Systems*, 2022, pp. 20147–20161.
- [36] J. Wang, Y. Hong, J. Wang, J. Xu, Y. Tang, Q.-L. Han, and J. Kurths, "Cooperative and competitive multi-agent systems: From optimization to games," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 5, pp. 763–783, 2022.
- [37] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multiagent reinforcement learning," *J. Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.
- [38] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" arXiv preprint arXiv: 2105.14491, 2021.
- [39] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," arXiv preprint arXiv: 1710.10903, 2017.
- [40] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, "Heterogeneous graph attention network," in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.
- [41] Y. Ye and S. Ji, "Sparse graph attention networks," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 1, pp. 905–916, 2021.
- [42] Z. Feng, D. Wu, M. Huang, and C. Yuen, "Graph attention-based reinforcement learning for trajectory design and resource assignment in multi-uav assisted communication," *IEEE Internet of Things J.*, 2024.
- [43] W. Du, T. Guo, J. Chen, B. Li, G. Zhu, and X. Cao, "Cooperative pursuit of unauthorized UAVs in urban airspace via multi-agent reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 128, p. 103122, 2021.
- [44] V. Isler, S. Kannan, and S. Khanna, "Randomized pursuit-evasion in a polygonal environment," *IEEE Trans. Robotics*, vol. 21, no. 5, pp. 875–884, 2005.
- [45] M. Riedmiller, R. Hafner, T. Lampe, M. Neunert, J. Degraeve, T. Wiele, V. Mnih, N. Heess, and J. T. Springenberg, "Learning by playing solving sparse reward tasks from scratch," in *Proc. Int. Conf. Machine Learning*, 2018, pp. 4344–4353.
- [46] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. AAAI Conf. Artificial Intelligence*, 2018, vol. 32, p. 1.
- [47] D. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *Proc. Int. Conf. Machine Learning*, 2015, pp. 1530–1538.
- [48] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

- [49] J. Liu, Y. Zhong, S. Hu, H. Fu, Q. Fu, X. Chang, and Y. Yang, "Maximum entropy heterogeneous-agent mirror learning," arXiv preprint arXiv: 2306.10715, 2023.
- [50] M. Wen, J. G. Kuba, R. Lin, W. Zhang, Y. Wen, J. Wang, and Y. Yang, "Multi-agent reinforcement learning is a sequence modeling problem," in *Proc. Advances in Neural Information Processing Systems*, 2022, pp. 16 509–16 521.



Zhe Peng received the B.Eng. degree in transportation from the School of Transportation and Logistics from Southwest Jiaotong University in 2022. He is currently a master student in transportation at the School of Traffic and Transportation Engineering, Central South University. His research interests include reinforcement learning and its applications in optimization, and control problems.



Guohua Wu (Senior Member, IEEE) received the B.S. degree in information systems and the Ph.D. degree in operations research from National University of Defense Technology, in 2008 and 2014, respectively. During 2012 and 2014, he was a visiting Ph.D. student at University of Alberta, Canada. He is currently a Professor at the School of Automation, Central South University. His current research interests include planning and scheduling, computational intelligence.

He has authored more than 100 referred papers and serves as an Associate

Editor of *Information Sciences*, and an Associate Editor of *Swarm and Evolutionary Computation Journal*.



Biao Luo (Senior Member, IEEE) received the Ph.D. degree in control science and engineering from Beihang University in 2014. He is currently a Professor with the School of Automation, Central South University. Before that, he was an Associate Professor with the Institute of Automation, Chinese Academy of Sciences. His current research interests include intelligent control and decision-making.

He serves as an Associate Editor for *IEEE TNNLS*, *Artificial Intelligence Review*, and *Neurocomputing*. He is the Vice-Chair of Adaptive Dynamic Programming and Reinforcement Learning Technical Committee of CAA.



Ling Wang (Member, IEEE) received the B.Sc. degree in automation and the Ph.D. degree in control theory and control engineering from Tsinghua University in 1995 and 1999, respectively. He became a Full Professor with the Department of Automation, Tsinghua University in 2008. His current research interests include intelligent optimization and production scheduling.

Prof. Wang has authored five academic books and more than 300 refereed papers. He was a recipient of the National Natural Science Fund for Distinguished Young Scholars of China, the National Natural Science Award, and the Natural Science Award nominated by the Ministry of Education of China. He is the Editor-in-Chief of *International Journal of Automation and Control* and an Associate Editor of *IEEE TEVC*, *Swarm and Evolutionary Computation*, and *Expert Systems With Applications*.